



Computer Networks

计算机网络 (CST2398)

2025-2026 学年第二学期 (26-Spring)

计算机科学与技术学院 王洁

wangjie_tongji@tongji.edu.cn



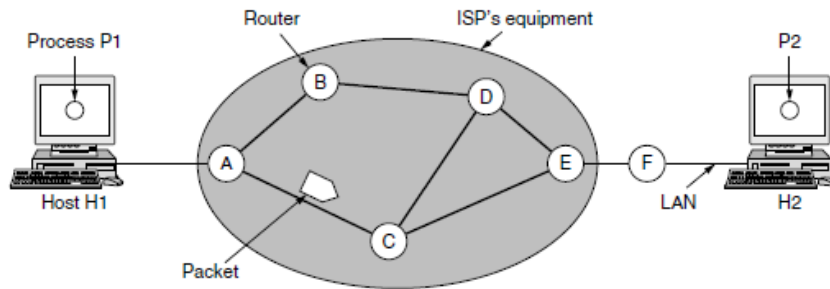
05

Network Layer

Contents of this Chapter

- **Design Issues of the *Network Layer*:**

- *Store-and-Forward Packet Switching*



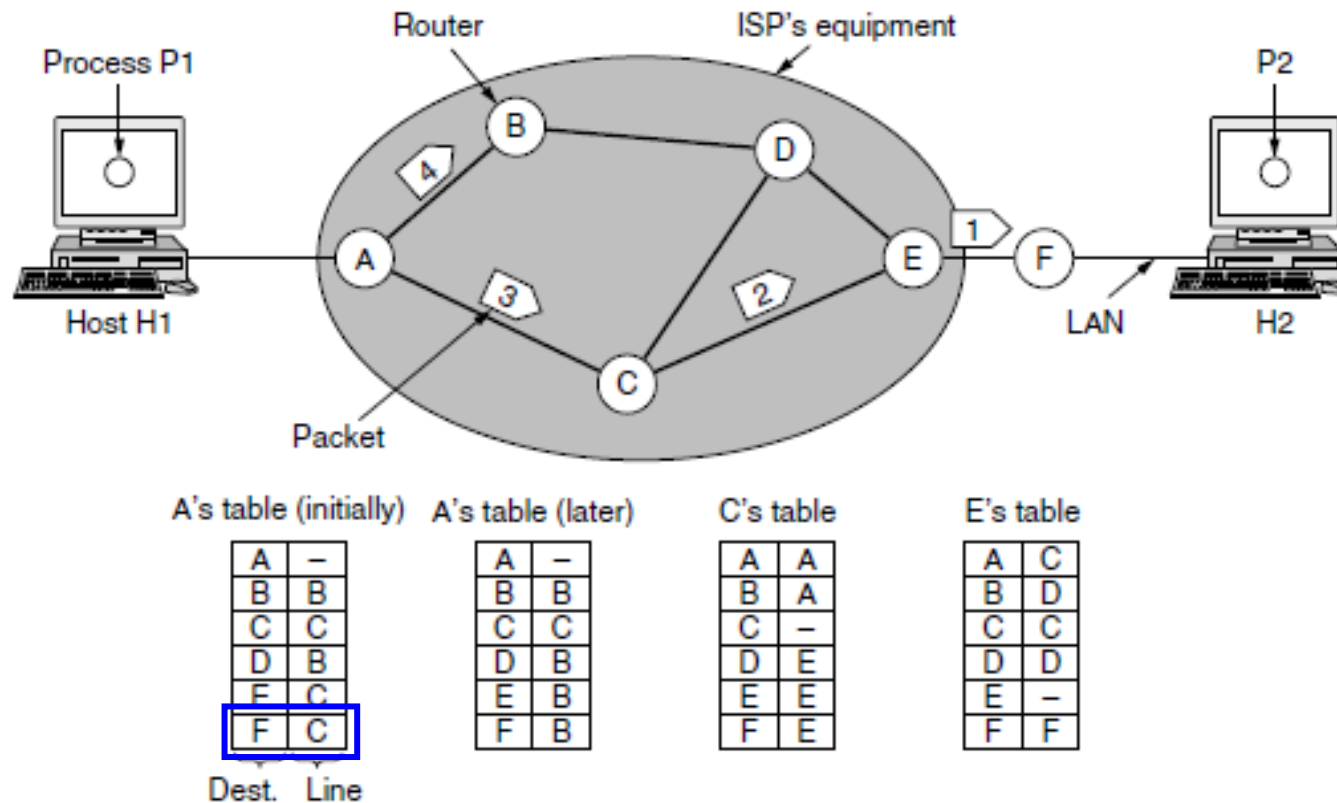
- **Addressing**: How to locate/number different (interfaces of) hosts and routers?
- **Routing**: How to find routes/paths for a packet from source to destination?
- **Congestion Control and QoS**

Application
Transport
<i>Network</i>
Link
Physical

Network: Routing of a packet from the source to the destination

Implementation of Connectionless Services

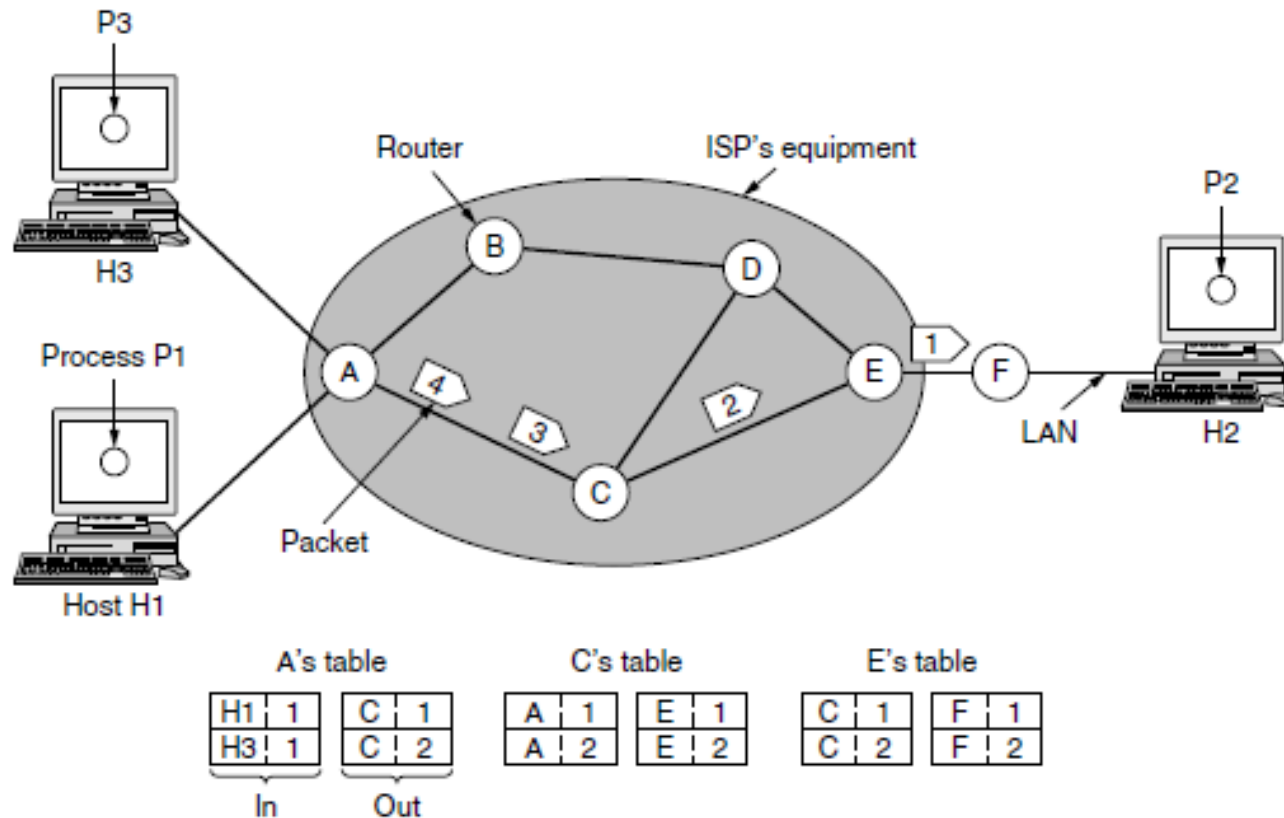
- **Connectionless: *Datagram* Network**



- Each packet/datagram is *routed individually*.
 - So they may go through different paths. E.g. P4 and P3
- Entry of the routing table:
 - *Destination*
 - *Outgoing line*
- E.g. IP (Internet Protocol)

Implementation of Connection-oriented Services

- **Connection-oriented: *Virtual Circuit***



- A *connection* is established.
During this connection,
 - Packets in the same traffic flow follow the same path.
E.g. P1, P2, P3, P4
 - Routing tables are fixed.
- Also called “label-switching”
 - ATM (Asynchronous Transfer Mode, 异步传输模式)
 - MPLS (Multiprotocol Label Switching, 多协议标签交换)

Comparison of the two services

- **Trade-offs**

Issue	Datagram network	Virtual-circuit network
Circuit setup	Not needed	Required
Addressing	Each packet contains the full source and destination address	Each packet contains a short VC number
State information	Routers do not hold state information about connections	Each VC requires router table space per connection
Routing	Each packet is routed independently	Route chosen when VC is set up; all packets follow it
Effect of router failures	None, except for packets lost during the crash	All VCs that passed through the failed router are terminated
Quality of service	Difficult	Easy if enough resources can be allocated in advance for each VC
Congestion control	Difficult	Easy if enough resources can be allocated in advance for each VC



5.1

Addressing and IP

Design Principles of Names and Addresses

1. No hardcoded addresses

- IP address can change, e.g., dial-in, mobile hosts

How about MAC addresses? Why are they different?

2. Single naming structure

- Addresses must be *unambiguous* -> can differentiate interfaces/routes
- Upper-layer protocols must be able to identify end-points unambiguously

3. Easy to process/compute

- Name should be case-insensitive ASCII -> easy to process
- Should be *easy to convert and compute* as binary in e.g., route computing

Binary computation,
AND, OR, XOR...

Internet Protocol (IPv4) Header

=4 32-bit words

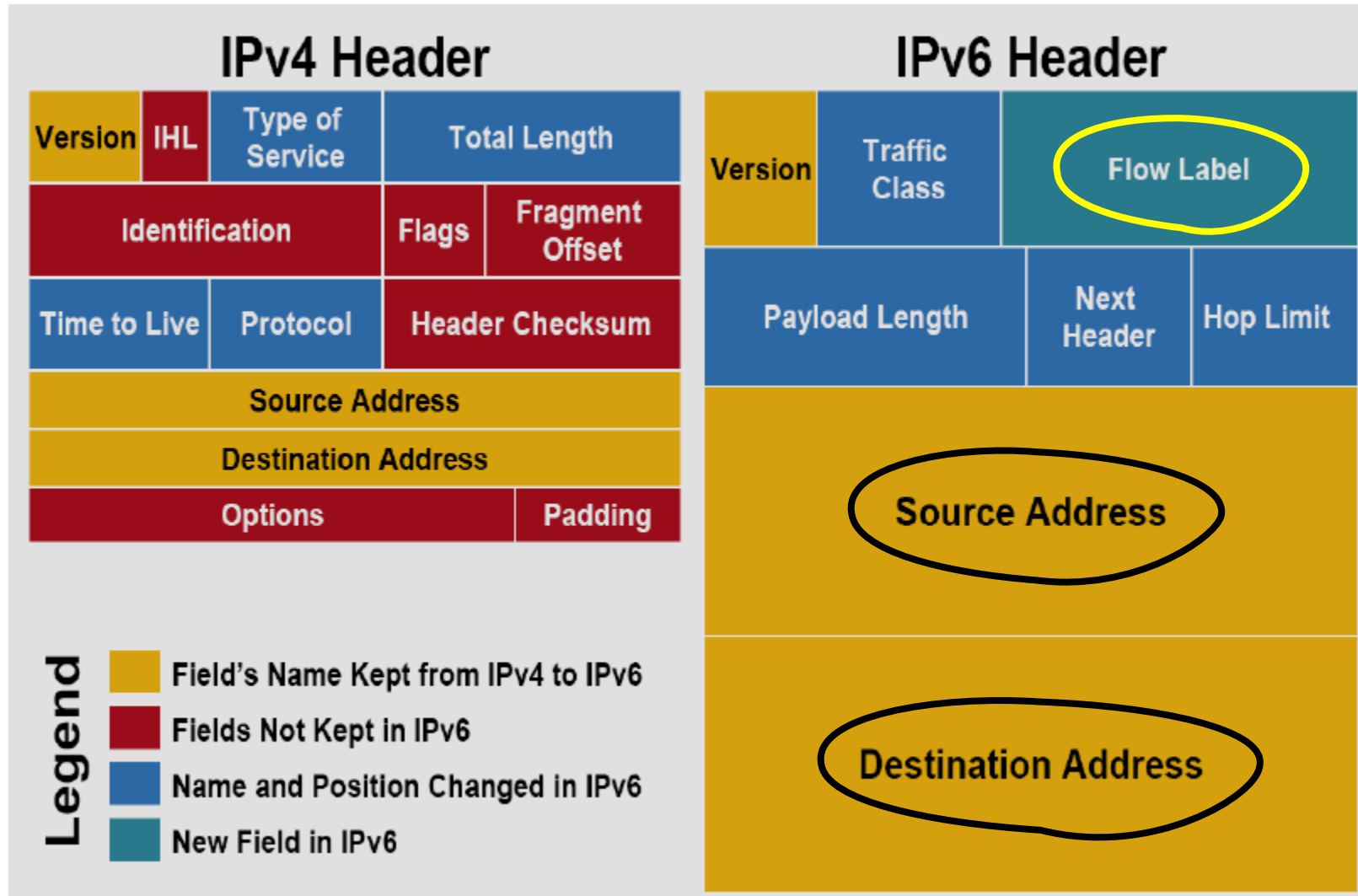
includes header and payload

Version (4)	Hdr Len (4)	TOS (8)	Total Length in bytes (16)	
Identification (16 bits)		Flags (3)	Fragment Offset (13)	
Time to Live (8)	Protocol (8)	Header Checksum (16)		
Source IP Address				
Destination IP Address				
Options (if any)			PAD	
Data Field				

- **Not all fields are used by “core” IP functions**

Context-less:
no state or
session info

IPv6 Header



- *Flow label*
- *Address length changes from 32 bit (IPv4) to 128 bit (IPv6).*

Internet (IPv4) Addresses

- **Format:** 32-bit, usually written in **dotted-decimal form**
- **Function:** *Specifies an interface (network connection), not a host!*
 - IP address may change.
 - A (multi-homed) host may have multiple addresses.
- **Hierarchical:** 2-level (initially) -> 3-level (later)
 - 2-level: network id + host id
 - 3-level: network id + subnet id + host id
- **Classful:** Class A, B, C (initially) -> CIDR (later)
 - Prefix 前缀 and mask 掩码

Special Types:

- (Sub-)network address: end with 0s.
- **Groupcast:** 224.0.0.0 - 239.255.255.255 (Class D addresses, start with 1110...)
- **Broadcast:** end with 1s.

Dotted Decimal Notation

Host/Domain Name	www.tongji.edu.cn
32-bit address	1100 1010 0111 1000 1011 1100 0110 0010
Dotted decimal	202.120.188.98

- **1-to-1 mapping:** *dotted-decimal* <-> *32-bit binary*
- **Not (domain) name** – Naming is handled by:
 - *Domain Naming System (DNS)* – a “registry” system.

IP Forwarding – Routing/Forwarding Tables

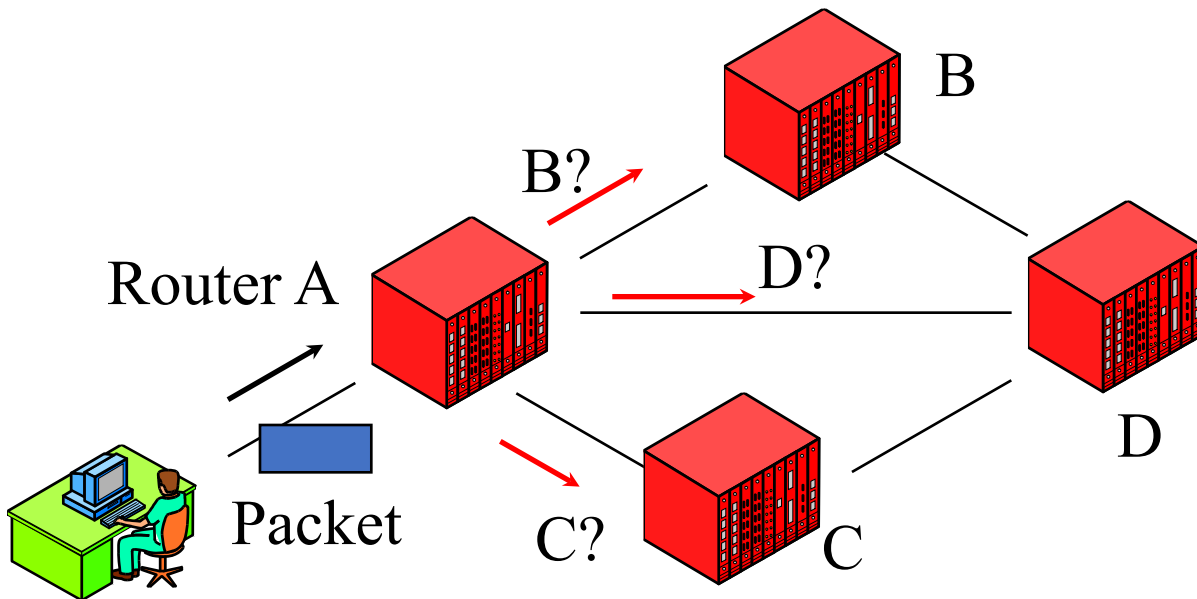
- **Rule:** Routers look at **destination IP address** of a packet to make decisions, and **nothing else!**

- **Routing table: a mapping**

- **Entry:** Destination IP address → outgoing link (which router to forward to)

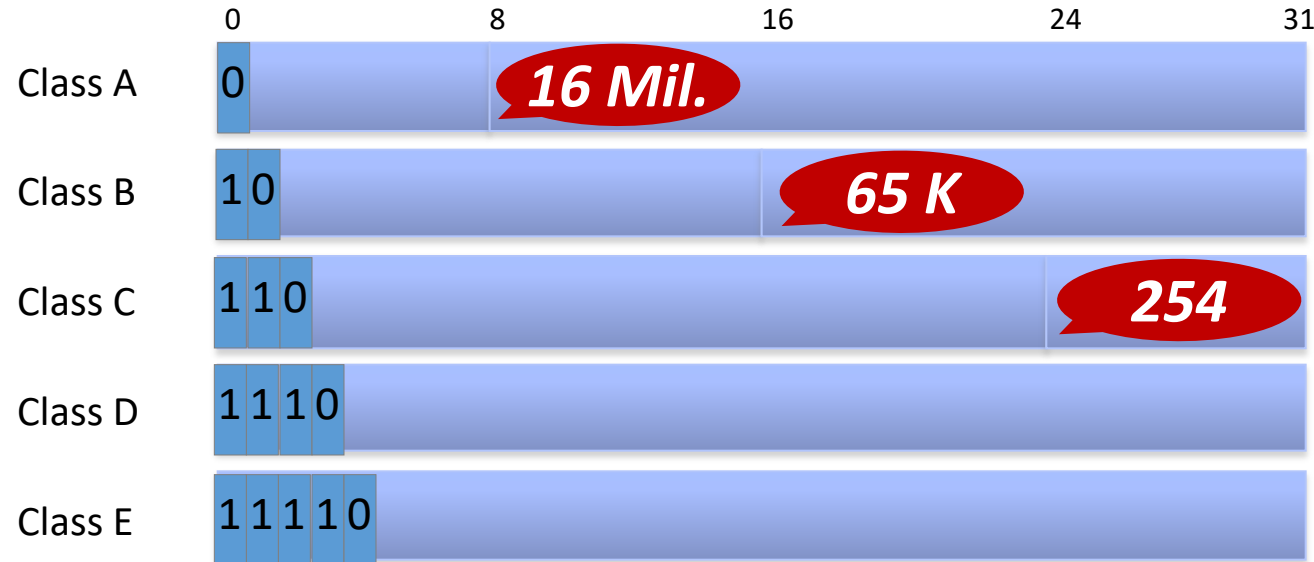
Mapping

- **Indexing:** **the network part** of the destination IP address



Network/Subnet	Delivery
128.171.17.	Local
142.99.	Next-Hop Router A

Classes of IP Addresses (history)



Why $2^n - 2$?
Hint

Internet (IPv4) Addresses

- Format:** 32-bit, usually written in **dotted-decimal form**
- Function:** Specifies **an interface (network connection)**, not a host!
 - IP address may change.
 - A (multi-homed) host may have multiple addresses.
- Hierarchical:** 2-level (initially) -> 3-level (later)
 - 2-level: network id + host id
 - 3-level: network id + subnet id + host id
- Classful:** Class A, B, C (initially) -> CIDR (later)
 - Prefix 前缀 and mask 掩码

Special Types:

- [Sub-network address: end with 0s.
- Groupcast: 224.0.0.0 - 239.255.255.255 (Class D addresses, start with 1110..)
- Broadcast: end with 1s.

Computer Networks (CST2398) - Copyright 王洁 2026/4/23 11

Class	First Byte	# Networks	Hosts per Network	Comments
A	< 128	2^7 (128)	$2^{24} - 2$ (16M)	Mostly used
B	128...191	2^{14} (16384)	$2^{16} - 2$ (65534)	Mostly used
C	192...223	2^{21} (2M)	$2^8 - 2$ (254)	
D	224...239	2^{28} (268M)	-	Dynamic, multicast
E	240...255	2^{27} (134M)	-	reserved

Not fair to
new comers

Problems in Address Classes

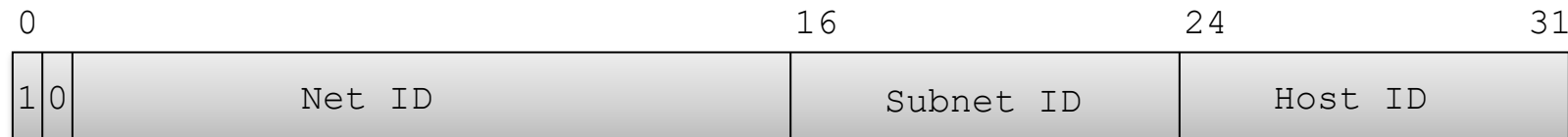
- **Large number of networks -> increasing routing table size**
 - Temporary fix: the same IP prefix shared by multiple physical networks.
 - “subnetting”
- **Exhaustion of Class B Addresses**
 - Temporary fix: use multiple class C addresses instead.
 - “supernetting”
- **Physical, or Administrative?**
- **Generalization: to de-couple the network size from IP itself**
 - Class Inter-Domain Routing (CIDR)



Solutions must
be *backward
compatible!*

Subnet Addressing

- **RFC 950, RFC 1122 – host must support this.**
 - *E.g. Large organization with multiple LANs. Class B block example:*

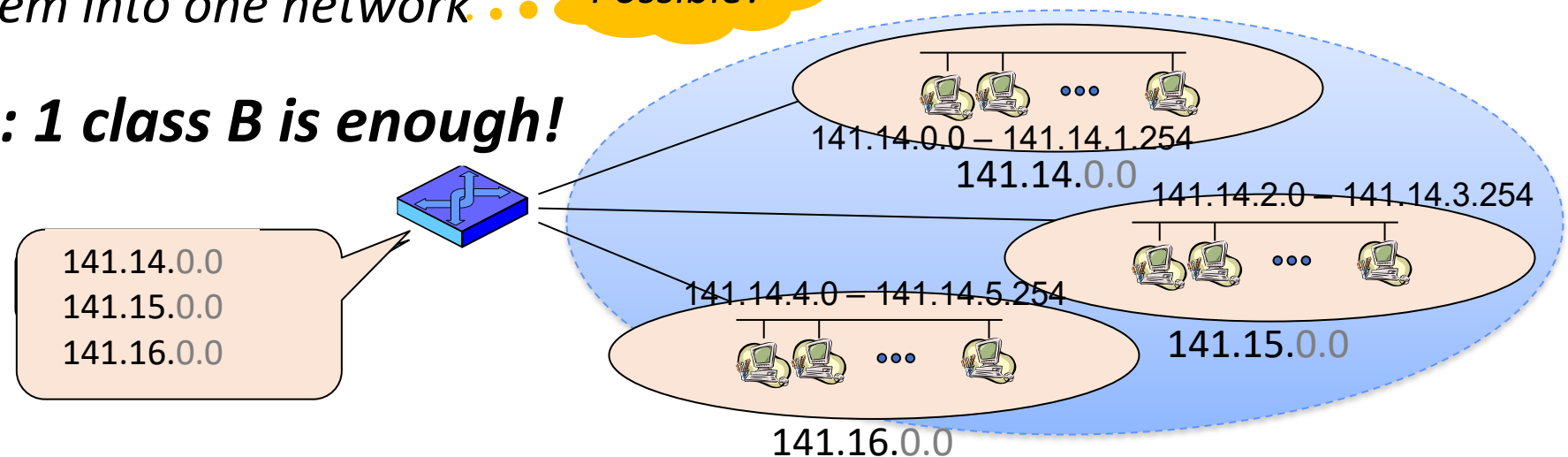


- **Result: 3-level hierarchical addressing**
 - *Hides internal network structure from the outside world. -> good for admin*
 - *Does not increase the size of routing tables.*
 - *Helps with address space exhaustion.*

A Subnetting Example

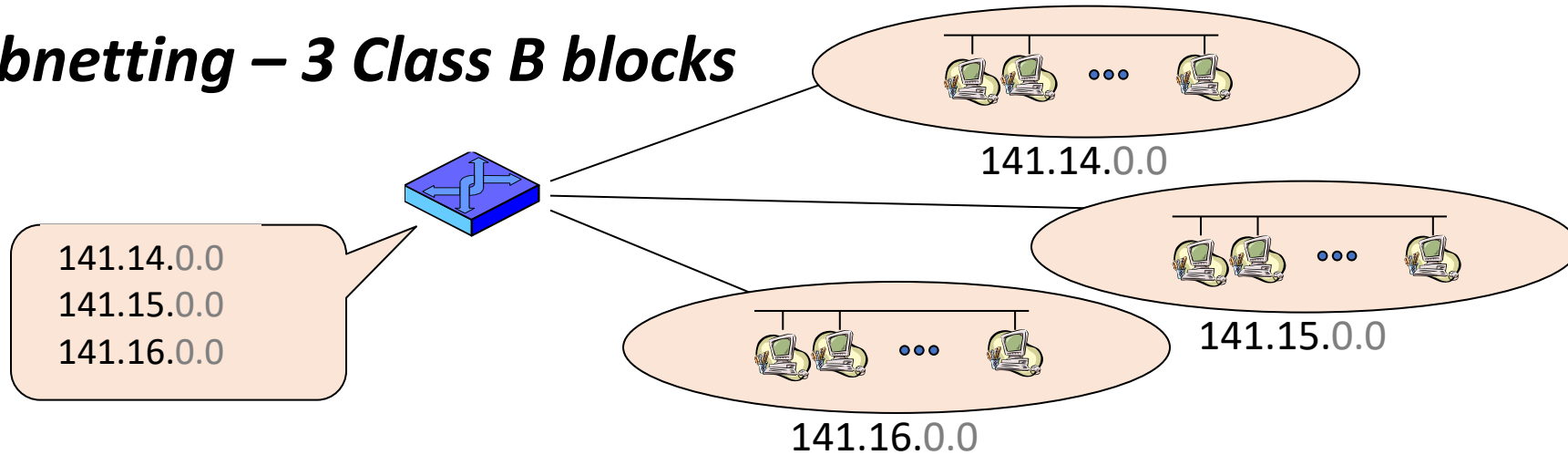
- **An organization with 3 physical networks**
 - Each is too big for a Class C, but together smaller than a Class B
- **W/o subnetting, this organization must**
 - Purchase 3 class B address spaces ... *\$\$ --*
 - Or connect them into one network ... *Possible?*
- **W/ subnetting: 1 class B is enough!**

How?

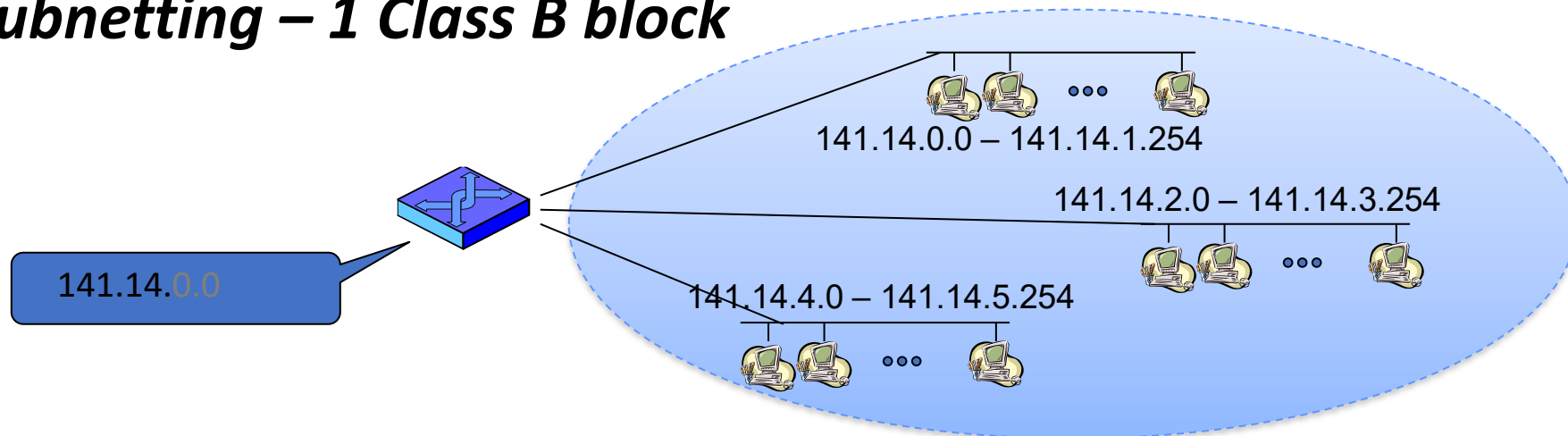


A Subnetting Example (Cont'd)

- **No Subnetting – 3 Class B blocks**

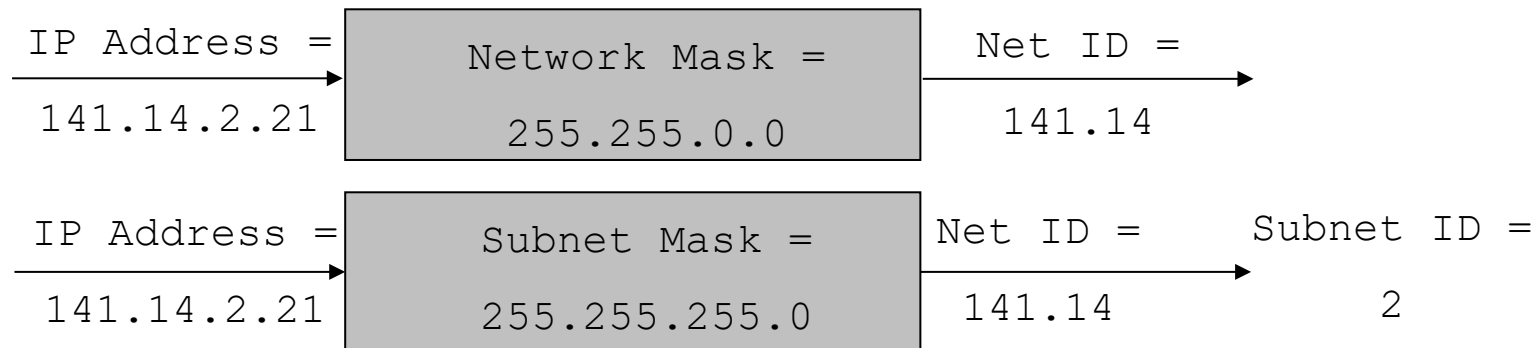


- **With Subnetting – 1 Class B block**



Subnet Masks 子网掩码

- **Masking:** the process of extracting the address of the *physical network* from an IP address
- **Mask: a 32-bit number. Bits are set to:**
 - 1, if this bit is a part of the (sub-)network address.
 - 0, if this bit is a part of the host address.
- **Subnet mask -> Subnet id, Network mask -> Network id**



How does the mask work: Bitwise-AND

Dotted decimal IP address

32-bit IP address

Implied (Classful) "Mask"

Network address

Dotted decimal network address

152 . 1 . 54 . 48
1001 1000 0000 0001 0011 0110 0011 0000
1111 1111 1111 1111 0000 0000 0000 0000
1001 1000 0000 0001 0000 0000 0000 0000
152 . 1 . 0 . 0

Dotted decimal IP address

32-bit IP address

Subnet Mask

Subnetwork address

Dotted decimal subnet address

152 . 1 . 54 . 48
1001 1000 0000 0001 0011 0110 0011 0000
1111 1111 1111 1111 1111 1111 0000 0000
1001 1000 0000 0001 0011 0110 0000 0000
152 . 1 . 54 . 0

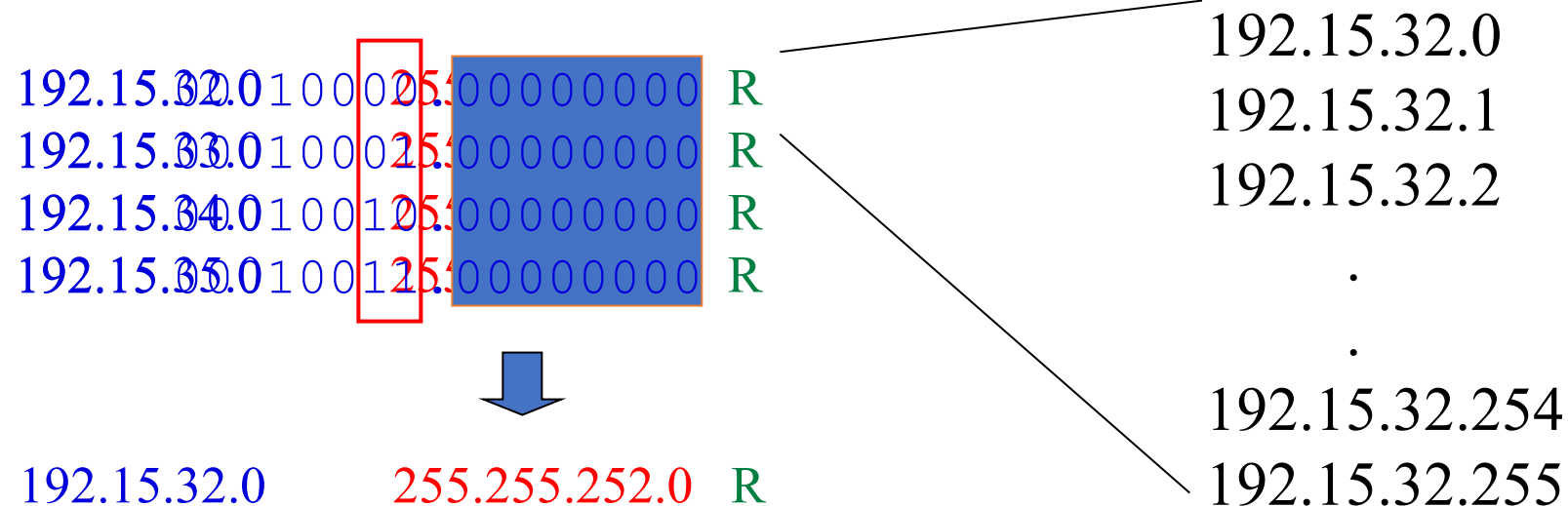
Forwarding Tables with Masks

- **Enabling subnetting:** add a “*mask*” column in the table
- **In reality:** destination and mask are both 32-bit number
- **E.g.** Destination address 128.171.17.13, subnet mask /24
 - 24 1's in the subnet mask -> Router only cares about 128.171.17.0
 - It matches the first row's network/subnet bits, so deliver to the local network.

Net/Subnet	Mask (# of 1's)	Destination
128.171.17.0	24	Local
142.0.0.0	8	Next-Hop Router A

Supernetting

- **Why**: Exhaustion of Class B addresses. Routing table size explosion.
- **Main idea**: *aggregation* of 2^n contiguous Class C addresses.
 - E.g. 2000 addresses needed -> allocate a block of 2048 addresses (8 Class C blocks)
- **Routing**: aggregate different routes -> flexible masking.



Classless Inter-Domain Routing (CIDR)

- **Why**: aggregate routes to save space
- **How**: collapse *a block of contiguous addresses* into a single logical network.
 - Not restricted to Class C addresses -> arbitrary sizes
 - Entry of the routing table: (network address, mask)
 - 32-bit Network address: the smallest address in the block
 - 32-bit Mask: can derive the number of addresses in this network

A "Cinderella" solution

$$(248)_{10} = (11111000)_2$$

Site	Lowest	Highest	Mask	Total #
1	194.24.0.0	194.24.7.255	255.255.248.0	$2^{11} = 2048$
2	194.24.8.0	194.24.11.255	255.255.252.0	?
3	194.24.16.0	194.24.31.255	255.255.240.0	?

Try it

An Example of CIDR

Network Address	Mask	Next Hop
11000010 00011000 00000000 00000000	11111111 11111111 11111000 00000000	R1
11000010 00011000 00001000 00000000	11111111 11111111 11111100 00000000	R2
11000010 00011000 00010000 00000000	11111111 11111111 11110000 00000000	R3

- Packet comes in addressed to 194.24.17.4:
- 11000010 00011000 00010001 00000100
- Boolean ANDed with Site 1 mask =
- 11000010 00011000 00010000 00000000
 - does not match Site 1 base address
- Boolean ANDed with Site 2 mask =
- 11000010 00011000 00010000 00000000
 - does not match Site 2 base address
- Boolean ANDed with Site 3 mask =
- 11000010 00011000 00010000 00000000
 - matches Site 3 base address → sent to R3

Still a short-term solution. What is the long-term solution? IPv6

Default Routes

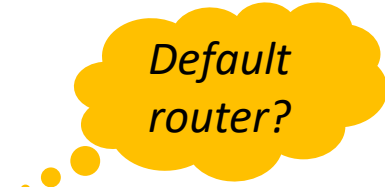
- **Problem: Too many networks/destinations in the network**
 - *Technically, need a next-hop information for every network in the Internet.*
- **An “Engineering” Solution**
 - *Keep detailed routes only for local neighborhood.*
 - *Use a **default** route/next-hop for **unknown destinations**.*
- **Results:**
 - *Saves space in the routing table.*
 - *But may select non-optimal paths.*

Forwarding Table Selection/Matching Rules

- Compare destination IP address of an arriving packet against **ALL** rows.

- Select a single row that matches.
- If multiple rows match, select the longest match. E.g., Line 3 and Line 5.
- If there is no match, select the default row.

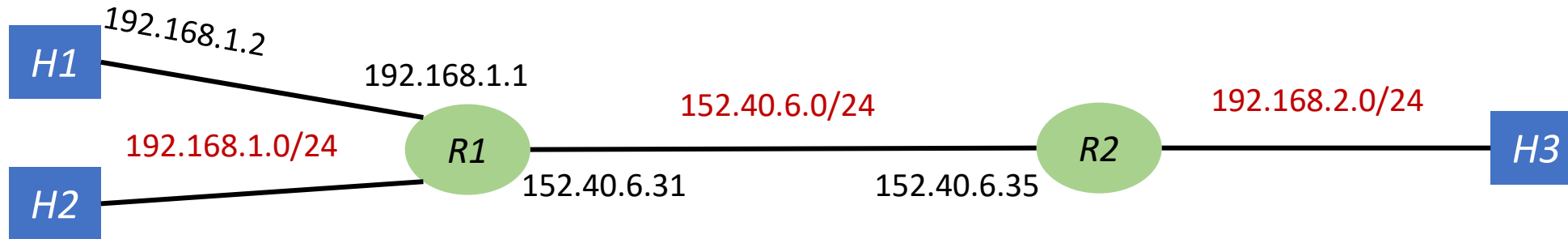
Line	Destination Address	Netmask	Interface	Next-Hop Router
1	152.19.0.0	16	2	B
2	152.15.33.0	24	1	Local
3	152.40.0.0	16	2	B
4	152.229.0.0	16	1	D
5	152.40.6.0	24	3	E
6	0.0.0.0	0	3	H



Default
router?

Ex.5.1 Configuration and Forwarding Table

A Simple Static Configuration Example



Configuration of H1

- IP address: 192.168.1.2
- Subnet Mask: 255.255.255.0
- Default Gateway: 192.168.1.1

convention

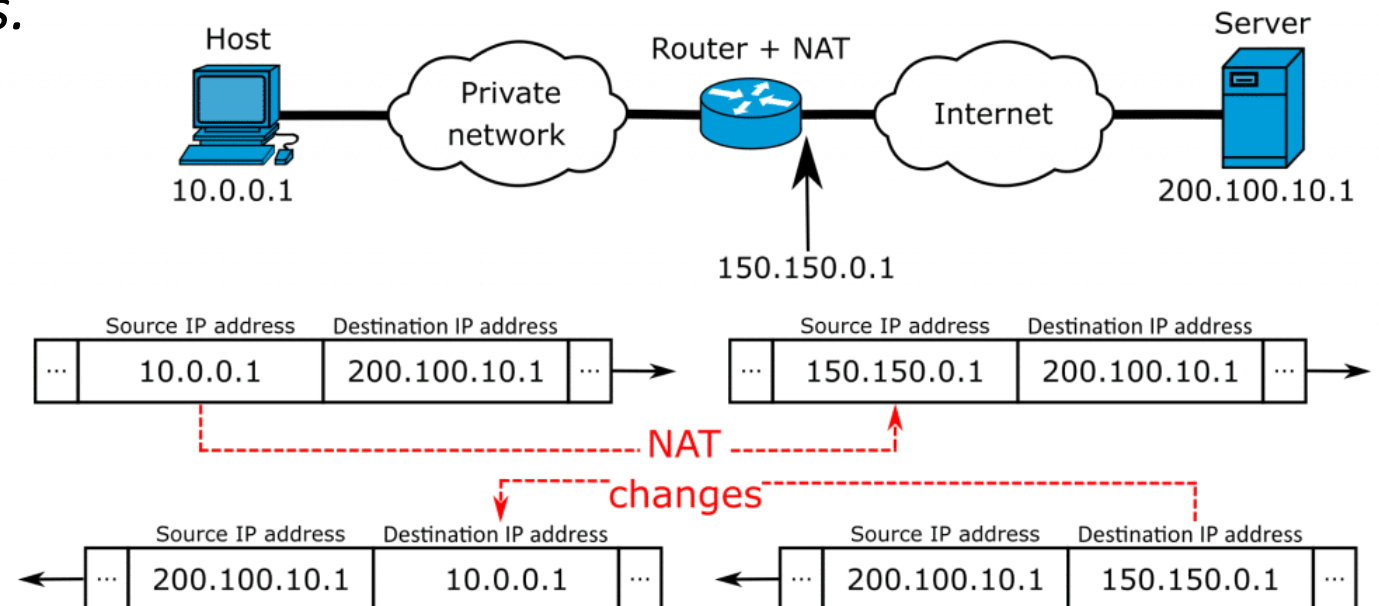
Forwarding Table at R1

Destination	Mask	Next-Hop
192.168.1.0	255.255.255.0	direct
152.40.6.0	255.255.255.0	direct
0.0.0.0	0.0.0.0	152.40.6.35

What is a possible configuration for H2, H3? Routing table at R2?

Aiding Protocols for IP - NAT

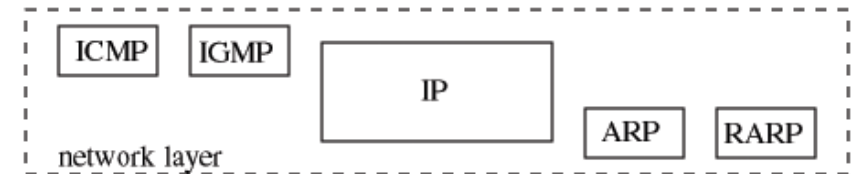
- **NAT (RFC2663): Network Address Translation**
 - (Try to) solve the problem of IPv4 address exhaustion.
 - Hide private network details.
 - Translation Rules
 - Outside to Inside:
Translate then route
 - Inside to Outside:
Route then translate
- **P(ortal)AT: One to Many.**



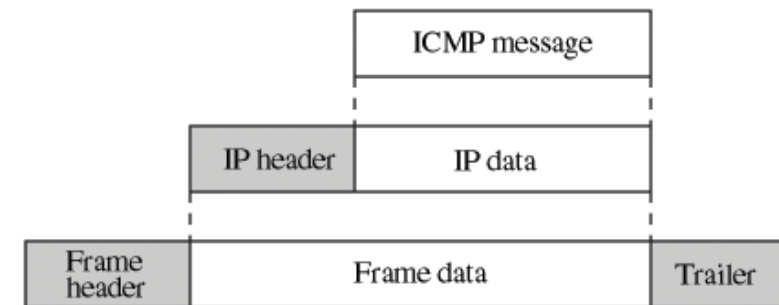
Aiding Protocols for IP - ICMP

- **ICMP (RFC792): Internet Control Message Protocol**

- *Communicate network-level errors or information about unexpected circumstances.*
- *Only report errors, no handling actions.*
- *Encapsulated in IP datagrams.*
- *E.g. ping (ECHO, ECHO REPLY)*
- *E.g. traceroute (TIME EXCEEDED)*



(a) Position of ICMP in the network layer

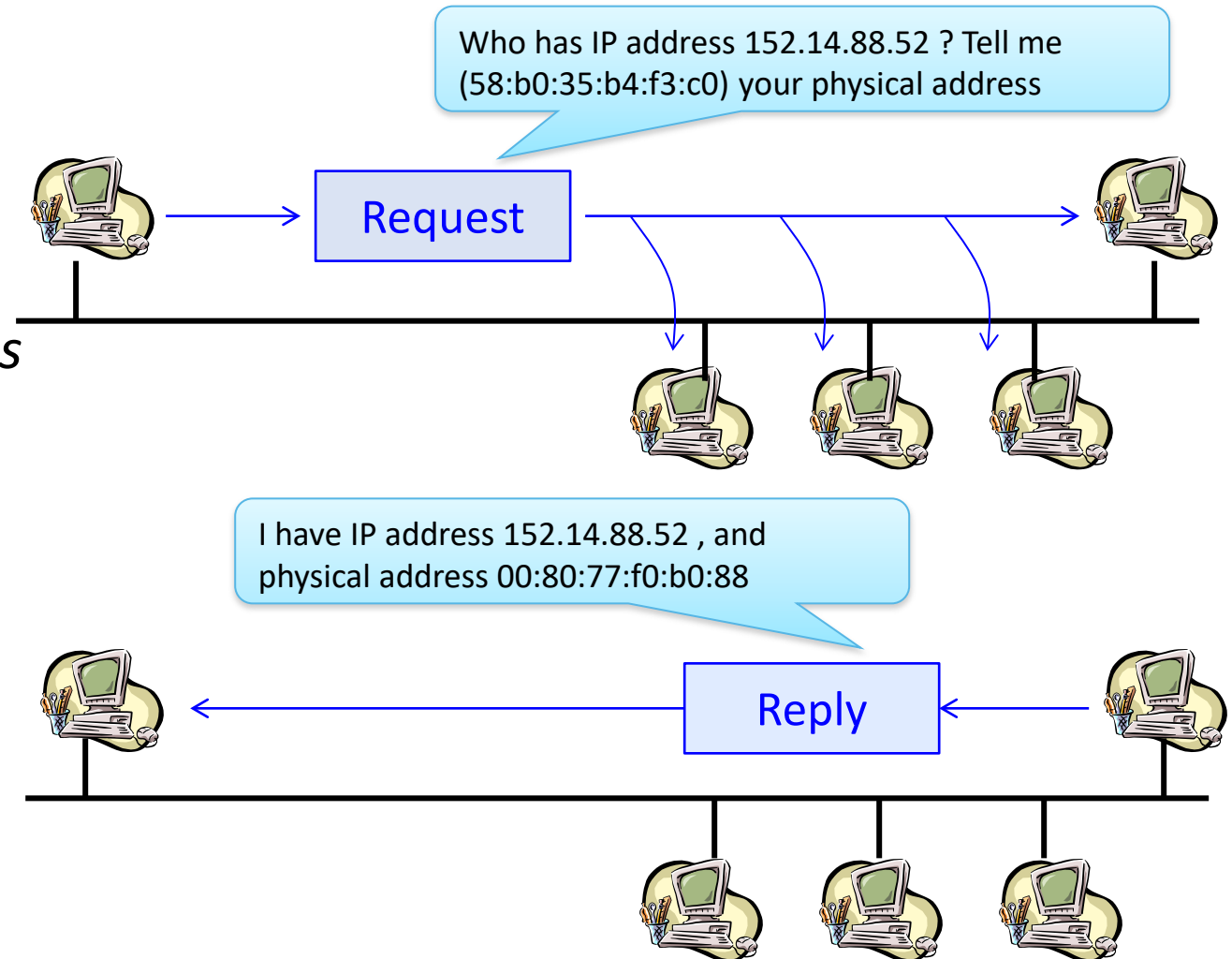


(b) ICMP encapsulation

Aiding Protocols for IP - ARP

- **ARP (RFC 826): Address Resolution Protocol**

- (Dynamically) mapping between IP and MAC addresses
- For broadcast networks, e.g. Ethernet, Token Ring.
- RARP: MAC -> IP
- ARP: IP -> MAC
- In IPv6: replaced by NDP



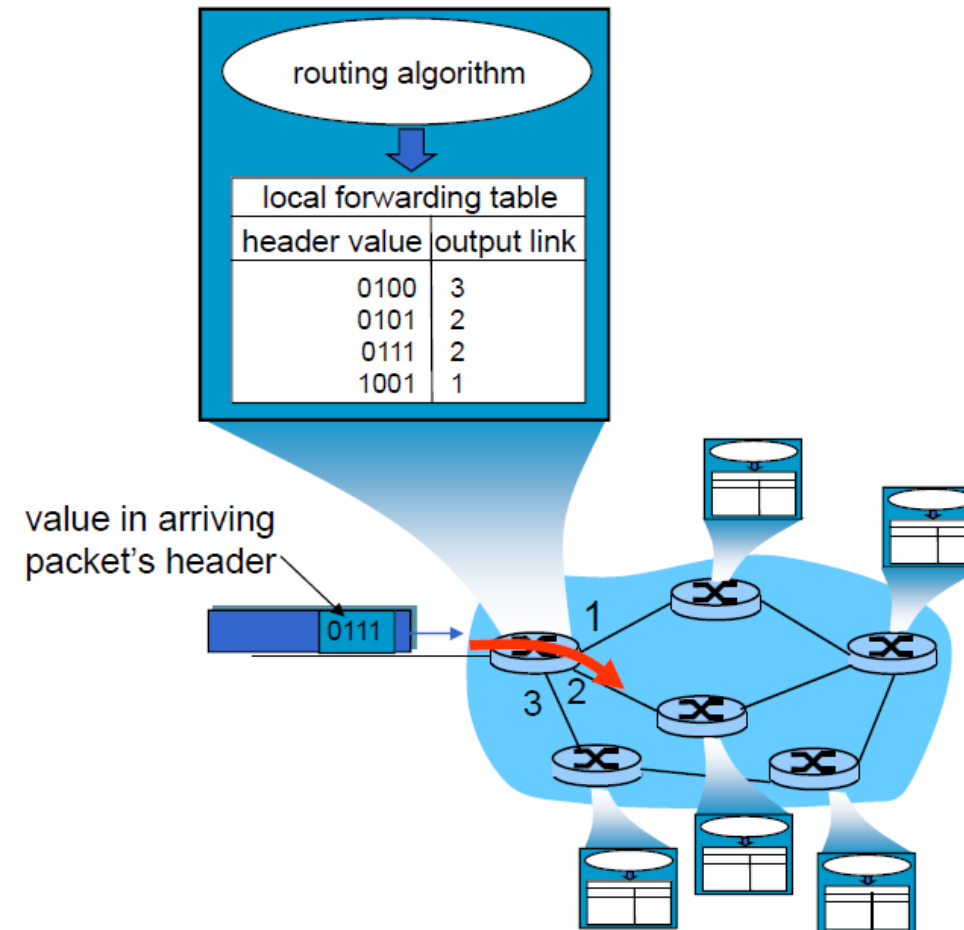


5.2

Routing Algorithms

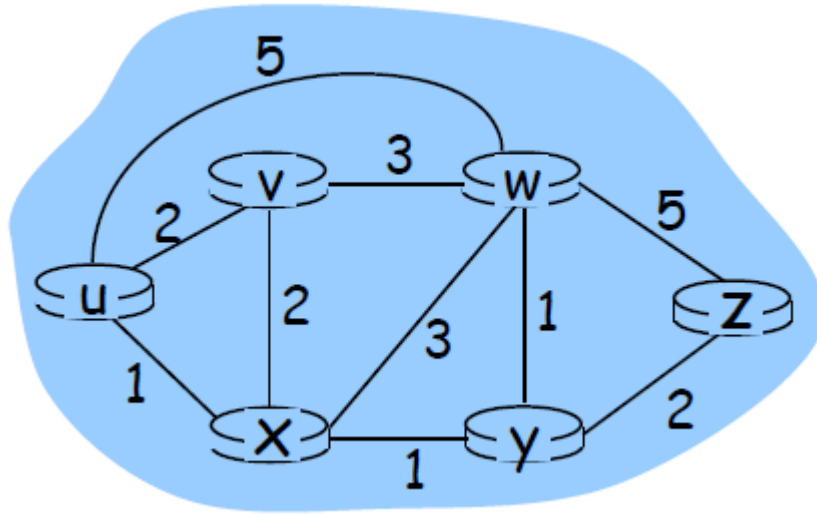
Routing, Forwarding, and Routing Algorithms

- **Routing:** Find the way for packets to go from source to destination
 - **Forwarding:** the action on packets taken by the router according to the routing/forwarding table
 - **Routing algorithm:** the algorithm to construct and update the routing table



Modeling of the Routing Problem

- **Graph Abstraction: Graph $G(N,E)$**



- **Routers $N = \{u, v, w, x, y, z\}$**
- **Links $E = \{(u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z)\}$**

direction?

- **Link Costs $l(u, v)$**

- $u, v \in N$ and $(u, v) \in E$
- $l: E \rightarrow \mathbb{R}^+$
- E.g. $l(u, v) = 2$

What can be defined as link cost?

Routing algorithm finds the shortest path

- **Length/Cost of a path**

$$(x_1, x_2, x_3, \dots, x_p) = l(x_1, x_2) + l(x_2, x_3) + \dots + l(x_{p-1}, x_p)$$

Classification of Routing Algorithms (1/2)

- **Dimension 1: Global information v.s. Decentralized information**

1) Global:

- All routers know:
 - The complete network topology $G(N, E)$;
 - Link costs of all links in E .
- “Link-state” algorithms
 - E.g. Dijkstra

2) Decentralized:

- A router i knows:
 - Its physically connected *neighbors* $\{j \mid (i, j) \in E\}$;
 - Link costs to its neighbors.
- Iterative process: exchange partial information w/ neighbors
- “Distance-vector” algorithms
 - E.g. Bellman-Ford

Classification of Routing Algorithms (2/2)

- **Dimension 2: Static v.s. Dynamic**

- *Routes: the path from a source to a destination*

1) **Static:**

- *Routes changes slowly over time, e.g. when network topology changes*
 - *a new router joins the network*
 - *a link is down/up*

2) **Dynamic:**

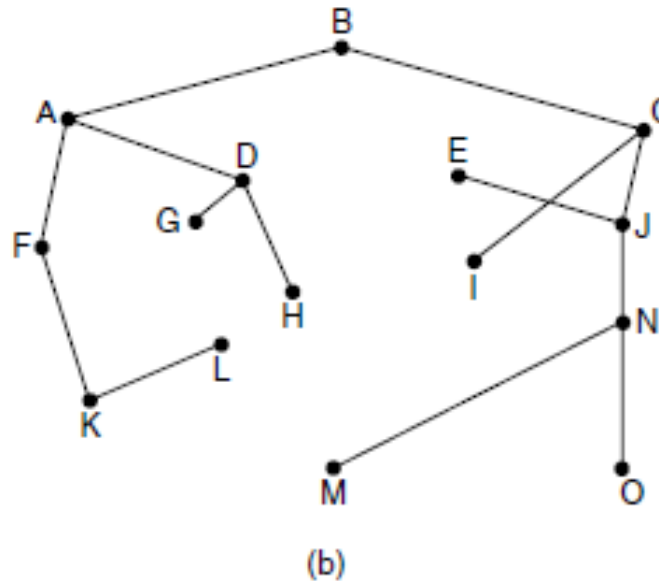
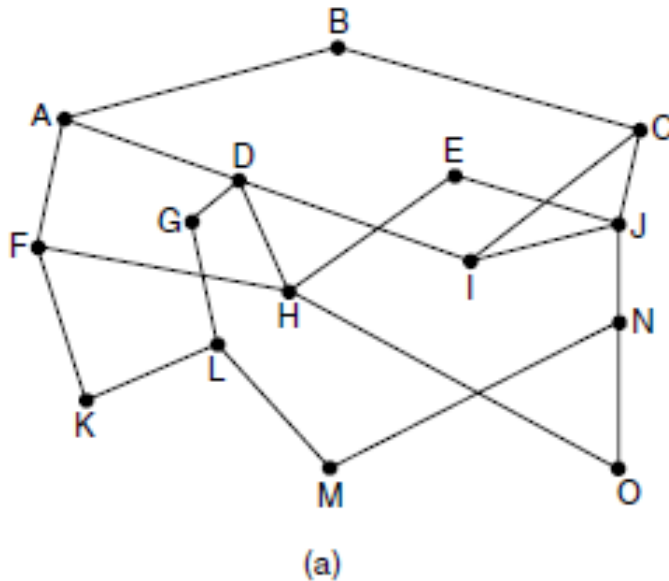
- *Routes changes quickly, e.g.*
 - *Periodic update*
 - *In response to link cost changes*

Routing Algorithms

- Shortest path routing: *Dijkstra's Algorithm*
- Distance vector routing: *Bellman-Ford Algorithm*
- Link State Routing
- Hierarchical Routing
- Broadcast Routing
- Routing for mobile hosts
- Routing in Ad Hoc Networks

The Optimality Principle

- *If router J is on the optimal (the shortest) path from routers I to K, then the optimal path from J to K also falls along the same route.*
- **Why:** proof by contradiction



Sink Tree: min. number of hops from the root to all nodes.

Sink Tree for Router B

Shortest Path Routing

- **Shortest:** *min. link cost*, so link cost needs to be
 - **Non-negative:** otherwise can not define “shortest”
 - **Additive:** cost of a path = $\sum$ cost of each link on the path
 - E.g. # of hops, avg. delay, geographical distance, ...
 - Note: A link can have different costs/lengths on different directions.
- **Shortest path routing algorithms** route each packet along a min. length (i.e. shortest) path between the source and destination nodes.
- **Static** routing algorithm

Dijkstra's Algorithm

- **Function:** Finds the shortest path from *a source* to *all other nodes* in the network.
- **Characteristics:**
 - Needs global topological information
 - All nodes/routers have the same information.
 - Done by “link state broadcast”.
 - Centralized computing of shortest paths
 - After k iterations, outputs the shortest path from the source to k (closest) destinations.

How? By building a shortest path tree (sink tree)



The Humble Programmer
by Edgar W. Dijkstra



For a network of N nodes, how many rounds are needed?

In each step/iteration: the closest node found in this step is added to a set A .

Algorithm Details

- **Notations:** for source node s , let

- N : the set of nodes whose shortest path is known.
- $l(i, j)$: link cost from node i to j . $l(i, j) = \infty$ if there is no direct link between i and j .
- $D(v)$: the current cost value of the path from source to the destination v .
- $p(v)$: the predecessor node (a neighbor of v) along the path from the source to node v .

- **Algorithm: 2-step**

1. **Initialization:** $N = \{s\}$; set $D(v) = l(s, v)$ for any node $v \notin N$
2. **Repeat step:** For node $w \notin N$, whose $D(w)$ is a minimum, add node w to N . Then update $D(v)$ for all the remaining nodes that are not in N by computing

$$D(v) \leftarrow \min_{w \in N} \{D(w), D(w) + l(w, v)\}$$

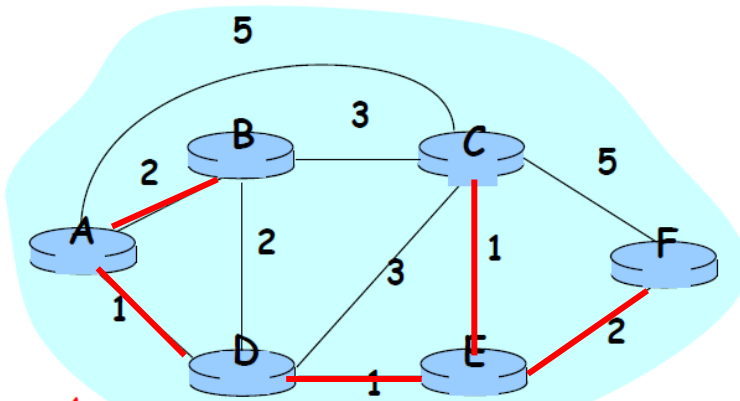
Dijkstra: Example

Step	Set N	D(B), p(B)	D(C), p(C)	D(D), p(D)	D(E), p(E)	D(F), p(F)
0	A	2, A	5, A	1, A	∞	∞
1	A, D	2, A	4, D	1, A	2, D	∞
2	A, D, B	2, A	4, D	1, A	2, D	∞
3	A, D, B, E	2, A	3, E	1, A	2, D	4, E
4	A, D, B, E, C	2, A	3, E	1, A	2, D	4, E
5	A, D, B, E, C, F	2, A	3, E	1, A	2, D	4, E

Resulting Forwarding Table at A

Dest.	Link
B	(A, B)
C	(A, D)
D	(A, D)
E	(A, D)
F	(A, D)

Tie: B and E



Source = A

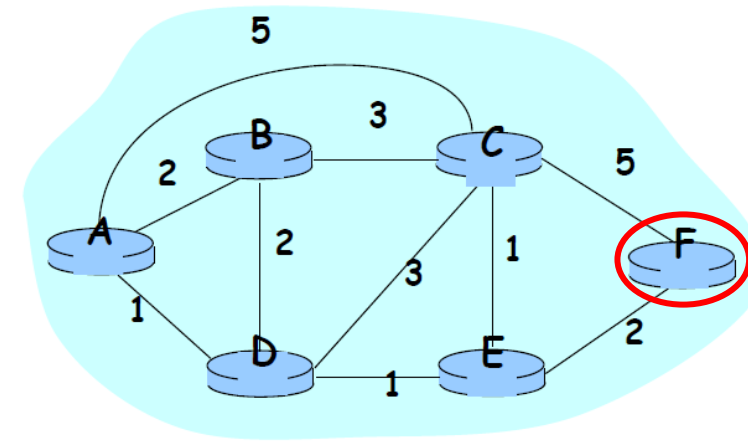
- Initialization:** $N = \{s\}$; set $D(v) = l(s, v)$ for any node $v \notin N$
- Repeat step:** For node $w \notin N$, whose $D(w)$ is a minimum, add node w to N . Then update $D(v)$ for all the remaining nodes that are not in N by computing

$$D(v) \leftarrow \min_{w \in N} \{D(v), D(w) + l(w, v)\}$$

Ex.5.2 Dijkstra's Algorithm

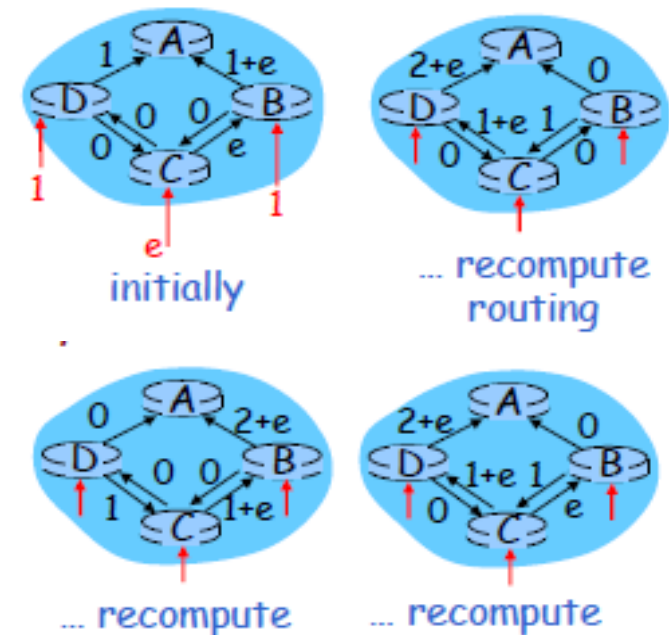
Find the forwarding table for Router F, using Dijkstra's Algorithm

Step	Set N	$D(B), p(B)$	$D(C), p(C)$	$D(D), p(D)$	$D(A), p(A)$	$D(E), p(E)$



Discussion on Dijkstra's Algorithm

- **Key Idea: iterating on the length of the path**
 - At the k -th iteration, set N contains k closest nodes to the source.
- **Algorithm complexity:** $O(n^2)$ or $O(n \log n)$ w/ improvements
 - Each iteration: check all nodes that are not in N
 - Total comparison: $n(n+1)/2 \rightarrow O(n^2)$
- **Possible oscillations (震荡)**
 - E.g. link cost = the amount of traffic on the link



E.g. oscillation

Distance Vector Routing

- **Decentralized Computing**
 - Each router only knows (**local**) information about its neighbors.
- **Dynamic Routing Algorithm**
 - Routes are updated periodically.
- **E.g. Bellman-Ford Routing Algorithm**
 - Function: Finds the shortest path from all nodes to the destination d .
 - The algorithm ends with **every node v** has their distances ($D(v)$) to node d , and the corresponding next node ($n(v)$) along the shortest path to the destination d .

Bellman-Ford Algorithm Details

1. Initialization: Let d = detention node, $D(d) = 0$. For any other node $v \neq d$, write $D(v) = \infty$, $n(v) = \cdot$, and label it with $(\cdot, \infty)$

2. Repeat step: For each node $v \neq d$, do the following. For each neighbor w of v ,
Update $D(v)$ by

Loop
1

$$D(v) \leftarrow \min_{w, (w,v) \in E} \{D(w) + l(v, w)\}$$

Distance from
node v to w

Distance from w
to destination d

Is it closer to get
to d via w ?
Yes -> update
No -> do nothing

Update $n(v)$ with the neighbor that achieves the min. value

$$n(v) = \arg \min_w \{D(w) + l(v, w)\}$$

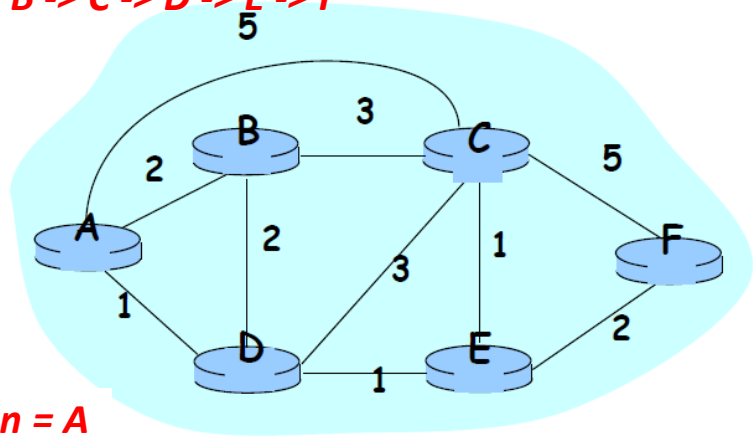
Repeat Step 2 until there is no further changes. . .

Converge!

Bellman-Ford: Example

Update Order:

$B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$



Cycle	$n(B), D(B)$	$n(C), D(C)$	$n(D), D(D)$	$n(E), D(E)$	$n(F), D(F)$
0	$(\cdot, \infty)$	$(\cdot, \infty)$	$(\cdot, \infty)$	$(\cdot, \infty)$	$(\cdot, \infty)$
1	$(A, 2)$	$(A, 5)$	$(A, 1)$	$(D, 2)$	$(4, E)$
2	$(A, 2)$	$(E, 3)$	$(A, 1)$	$(D, 2)$	$(4, E)$
3	$(A, 2)$	$(E, 3)$	$(A, 1)$	$(D, 2)$	$(4, E)$

Repeat step: For each node $v \neq d$, for each neighbor w of v ,

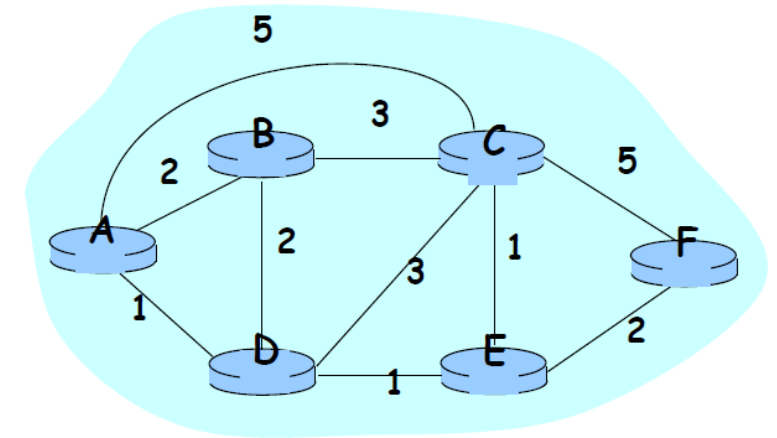
- Update $D(v)$ by $D(v) \leftarrow \min_{w, (w,v) \in E} \{D(w) + l(v, w)\}$
- Update $n(v)$ with the neighbor that achieves the min. value
$$n(v) = \arg \min_w \{D(w) + l(v, w)\}$$

Ex.5.3 Bellman-Ford Algorithm

Find the shortest paths to router E, using Bellman-Ford Algorithm.

- Update Order: B -> C -> D -> A -> F**

Cycle	$n(B), D(B)$	$n(C), D(C)$	$n(D), D(D)$	$n(A), D(A)$	$n(F), D(F)$
0	$(\cdot, \infty)$	$(\cdot, \infty)$	$(\cdot, \infty)$	$(\cdot, \infty)$	$(\cdot, \infty)$
1					
2					
3					
4					



The Count-to-Infinity Problem

- A Characteristic of Bellman-Ford Algorithm

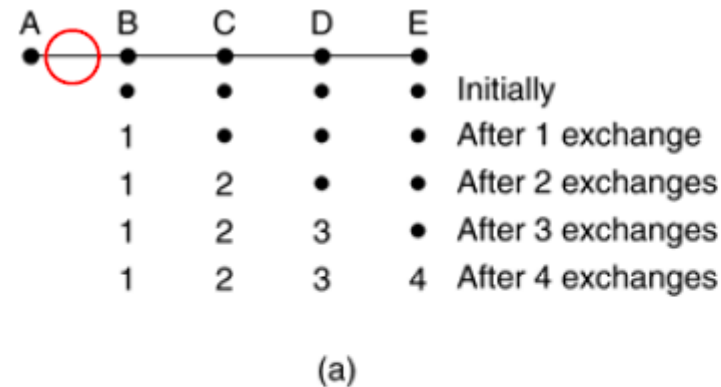
- **Count-to-Infinity**

- Good news travels fast.

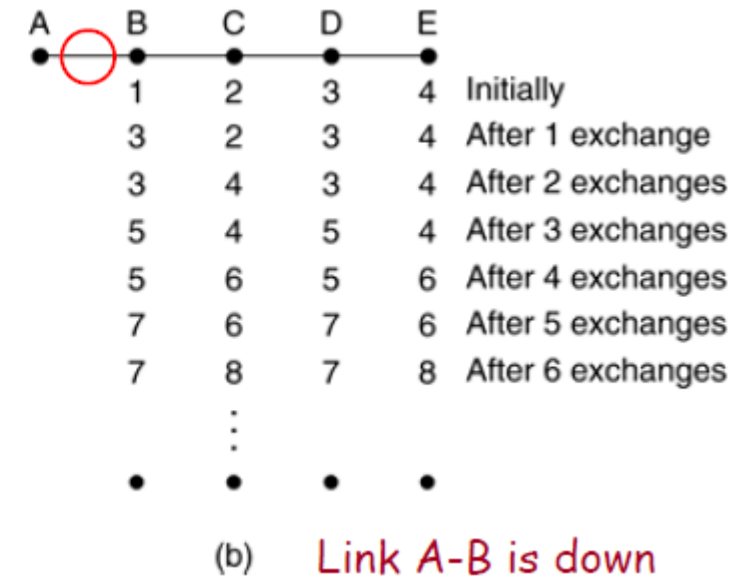
- **Bad news travels slow.**

- Consequences?

- E.g. cost metric: # of hops



Link A-B is up



Link A-B is down

E.g. the Count-to-Infinity problem

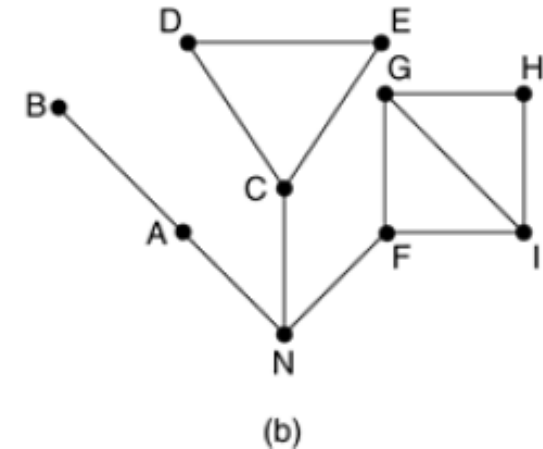
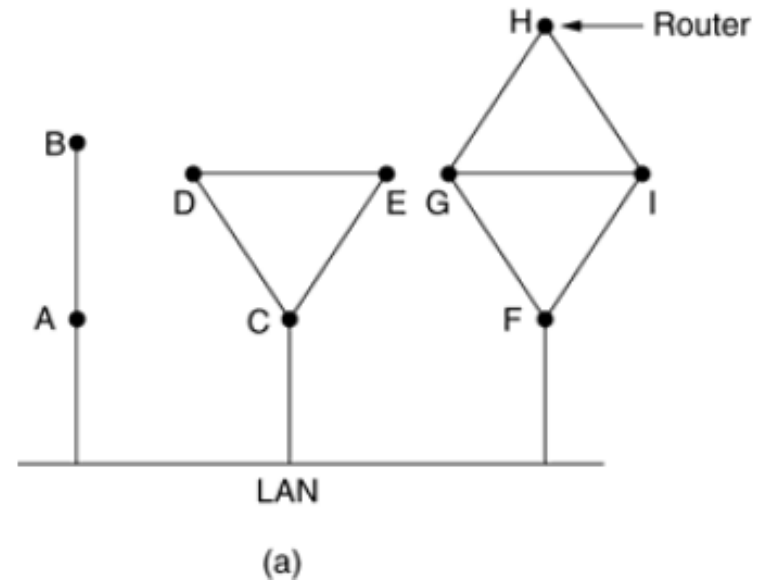
Link State Routing

- **How does the routing scheme (not only the algorithm) work?**
 - *How to learn the network topology (i.e. the graph)?*
 - *How to exchange necessary information?*
- **Each router does the following**
 1. *Discover its neighbors, learn their network address.*
 2. *Measure the delay or cost to each of its neighbors.*
 3. *Construct a packet telling all it has just learned.*
 4. *Send this packet to all other routers.*
 5. *Compute the shortest path to every other router.*

**Routing
Algorithm is here.**

1. Discovering Your Neighbors

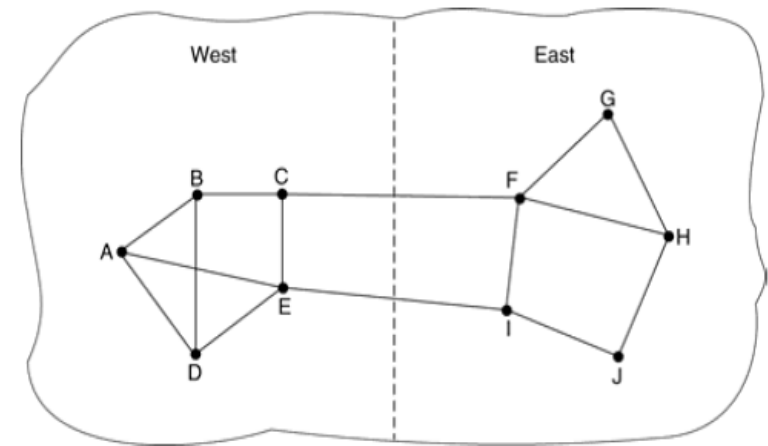
- *Send **HELLO** packet on each point-to-point link.*
- *Nodes who receive the **HELLO** packet will **reply with its address**.*



2. Measuring Link Costs

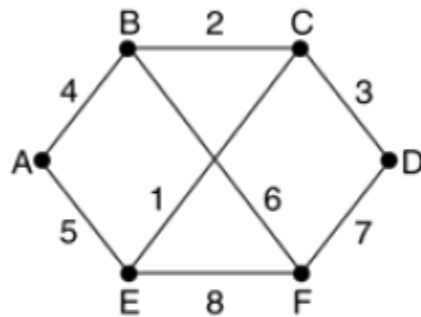
- *Send an **ECHO** packet over each link.*
- *Nodes who receives an **ECHO** packet is required to respond immediately.*
- *Measure the time it takes to complete this operation.*
 - *Which time should we measure?*
 - *Opinion 1: Only the transmission delay. To avoid oscillation between paths.*
 - *Opinion 2: All delays including queuing. More accurate to measure the entire delay.*

Link cost



3. Building Link State Packets

- The Link State packet consists of three parts.
 - Sequence Number (**Seq. #**): 32 bits, created for each packet.
 - **Age** (as a timer): decrease once (-1) per second.
 - **List of Neighbors** (with link costs).



(a)

(a) A subnet.

Link		State		Packets	
A	B	C	D	E	F
Seq.	Seq.	Seq.	Seq.	Seq.	Seq.
Age	Age	Age	Age	Age	Age
B 4	A 4	B 2	C 3	A 5	B 6
E 5	C 2	D 3	F 7	C 1	D 7
	F 6	E 1		F 8	E 8

(b)

(b) The link state packets for this subnet.

4. Distributing the Link State Packets

- **Flooding with refinements (Seq. # + Age)**
 - **Seq. #:** prevents duplicated packets from being propagated.
 - Lower Seq. # means older packet, and is thus rejected as obsolete.
 - **Age:** guarantees only live information are distributed.

Source	Seq.	Age	Send flags			ACK flags			Data
			A	C	F	A	C	F	
A	21	60	0	1	1	1	0	0	
F	21	60	1	1	0	0	0	1	
E	21	59	0	1	0	1	0	1	
C	20	60	1	0	1	0	1	0	
D	21	59	1	0	0	0	1	1	

The packet buffer for router B

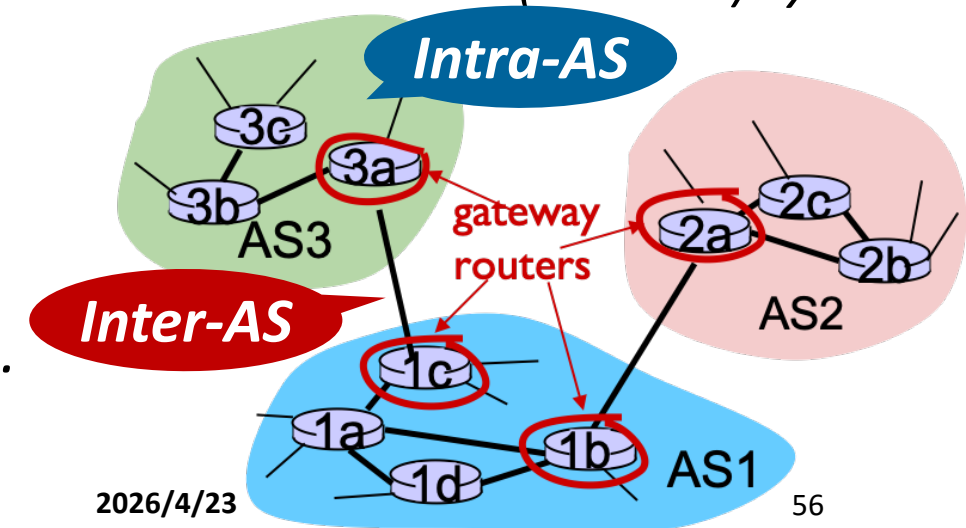
5. Compute the Shortest Path

- *Each router has a picture of the entire subnet graph*
- *Locally run a shortest-path algorithm on each router*
- *Dijkstra's algorithm*
- *Link-State routing is used in OSPF (in Internet) and ISIS*
 - *OSPF (Open Shortest Path First): widely used in the Internet*
 - *IS-IS: (Intermediate System – Intermediate System): similar to OSPF, earlier version of OSPF.*

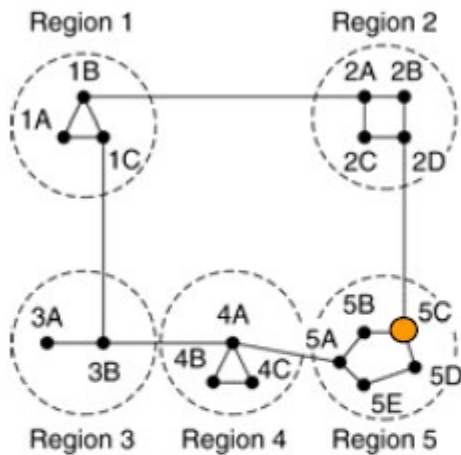
Hierarchical Routing - in Reality

- **Why: Routing table grows as network grows in size – design issues**
 - **Scalability**: overhead for computing, storing, and communicating the routing information
 - **Administrative autonomy**: who's in charge
- **How: Routers are organized into regions or autonomous systems (ASs):**
 - Routers within the same AS all run the same routing algorithm.
 - The routing algorithm running within an AS is called an intra-autonomous (**intra-AS**) system routing protocol (e.g., **RIP and OSPF**).

Gateway routers: Responsible for forwarding packets to destinations outside the AS, running inter-autonomous (**inter-AS**) system routing protocol (**BGP**).



An Example of Hierarchical Routing



Full table for 1A

Dest.	Line	Hops
1A	—	—
1B	1B	1
1C	1C	1
2A	1B	2
2B	1B	3
2C	1B	3
2D	1B	4
3A	1C	3
3B	1C	2
4A	1C	3
4B	1C	4
4C	1C	4
5A	1C	4
5B	1C	5
5C	1B	5
5D	1C	6
5E	1C	5

(b)

Hierarchical table for 1A

Dest.	Line	Hops
1A	—	—
1B	1B	1
1C	1C	1
2	1B	2
3	1C	2
4	1C	3
5	1C	4

4 hops to 5A

(c)

Multiple entries to the same AS are collapsed into one entry in the routing table.

Hierarchical routing.

Why different protocols?

1. Policy

- *Intra-AS: a single admin, so no policy decision is needed.*
- *Inter-AS: The admin for each AS wants control over how its traffic is routed.*

2. Scale

- *Hierarchical routing saves space (routing table size), reduces update traffic.*

3. Performance/Design issue

- *Intra-AS: can focus on performance*
- *Inter-AS: Policy may dominate over performance, e.g. security issues.*

Broadcast Routing

- *Send a separate packet to each destination*
- *Use flooding*
- *Use multi-destination routing*
 - *Each packet contains a list of destinations*
 - *Routers duplicate packet for all matching outgoing links*
- *Use **spanning tree** routing*
 - **Spanning tree**: *a subset of the subnet that includes all routers but contains no loops*
- *Reverse path forwarding*

Flooding

- **Force Flooding**

- *Every incoming packet is sent out on every outgoing line, except the one it arrived on.*
- *Generates a large number of duplicate packets.*
- *Need to limit multiple transmission (of the same packets).*

- **Selective Flooding**

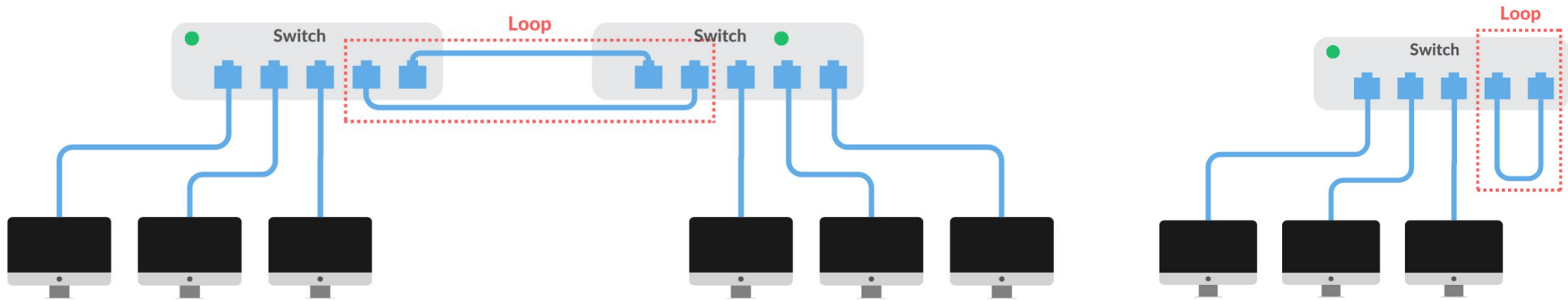
- *Flood only in the direction of the destination*

- **Practical in a few settings**

- *Military Applications*
- *Distributed Databases*
- *Metric for comparison*

What to avoid in Flooding: Broadcast Storm

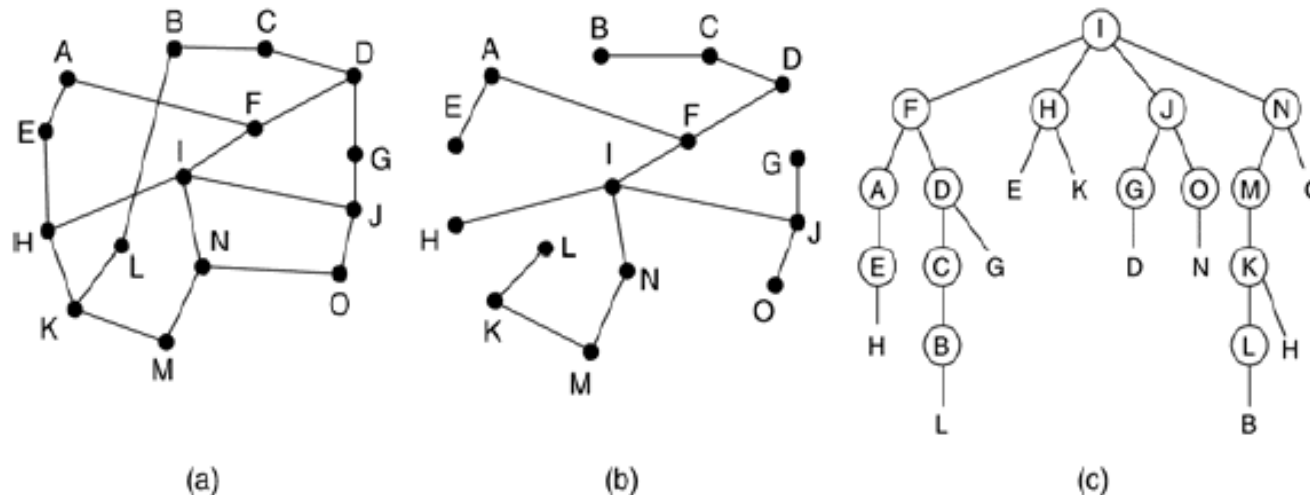
- **Possible reason1: Loops in the Network**



- **Possible reason2: The broadcast domain is too big.**
- **Possible reason3: A large number of IP address requests via DHCP.**

Spanning Tree Broadcasting

- **Idea**: Send the minimum number of packets necessary
- **Routers must be able to compute spanning tree**
 - Available with link state routing, not available with distance vector routing.



Why?

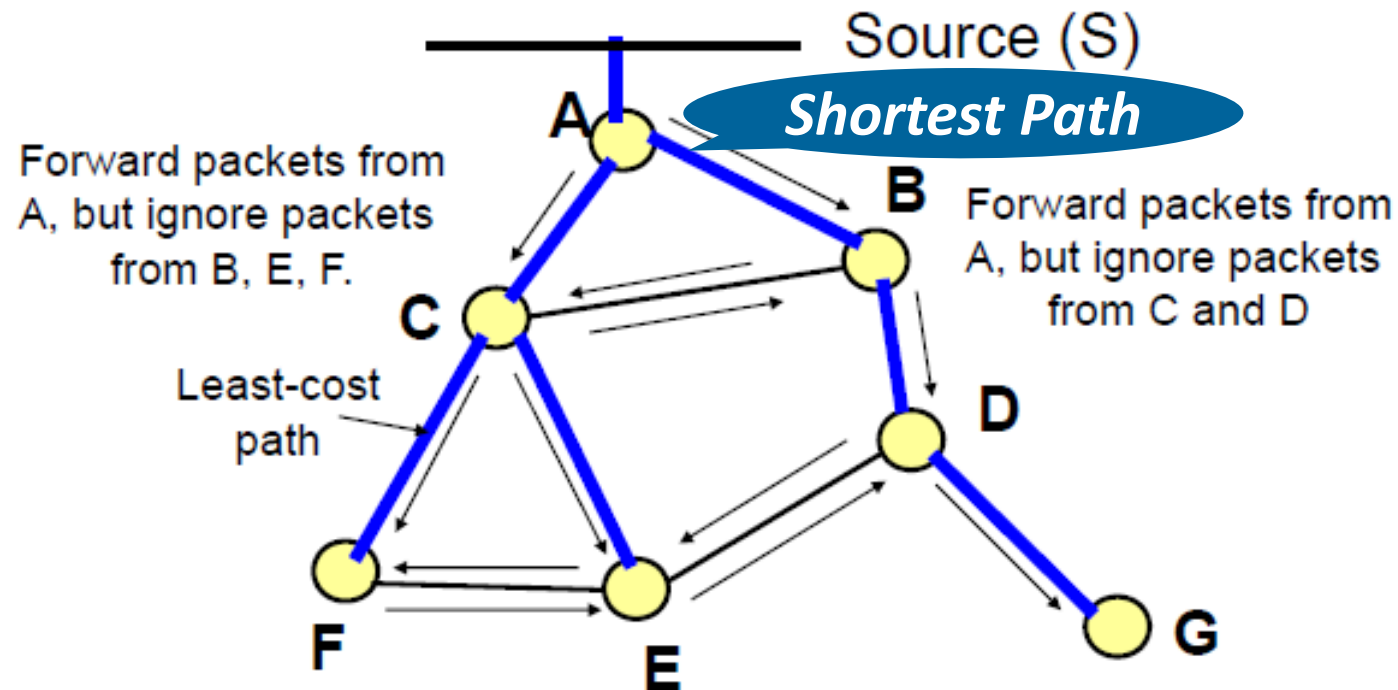
Reverse path forwarding. (a) A subnet. (b) A Sink tree (for I).
(c) The tree built by reverse path forwarding.

Reverse Path Forwarding

- **Why:** *knowledge of a spanning tree is not available*
- **Objective:** *to provide an approximation of spanning tree routing*
- **How:** *When a router receives a packet with a given source address, it **floods** the packet according to the following rules:*
 - *If the packet arrived on the link that is on its own shortest path back to the sender, the router duplicates the packet (and flood).*
 - *Otherwise, the router simply discards the packet without forwarding on any of its outgoing links. (because it is likely a duplicate)*

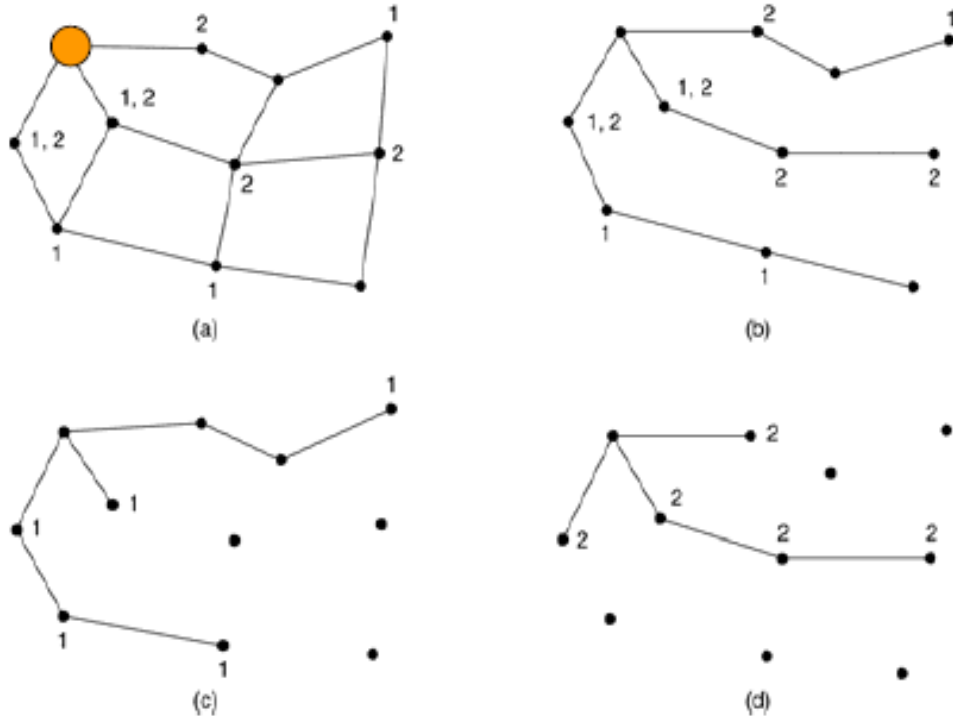
An Example of Reverse Path Forwarding

- *E.g. Node A broadcasts packets from source S.*



Multicast Routing

- **Multicast tree for a group**



(a) A network. (b) A spanning tree for the leftmost router.
(c) A multicast tree for group 1. (d) A multicast tree for group 2.

- **Used w/ Link State Routing**

The routers are aware of the network topology and the spanning tree can be computed.

Used w/ Distance Vector Routing

When a router receives a packet for a multicast group for which it has no subscribers (hosts or other routers), the router sends a PRUNE message to the source router.

Routing for Mobile Hosts

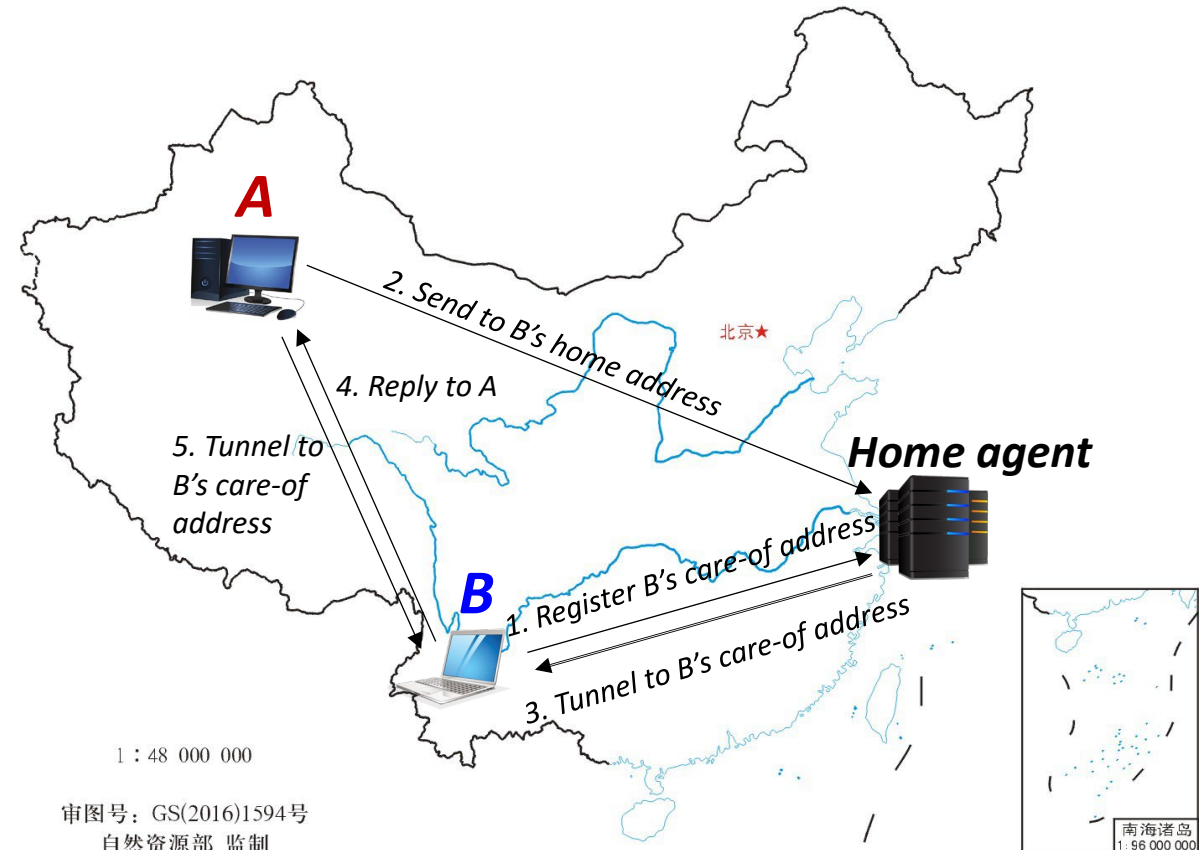
- **Tunneling 隧道**

- Home Address -> Home Agent
- Local Address (Care of address)

- **NA(P)T: Network Address (Ports) Translation (RFC 3022)**

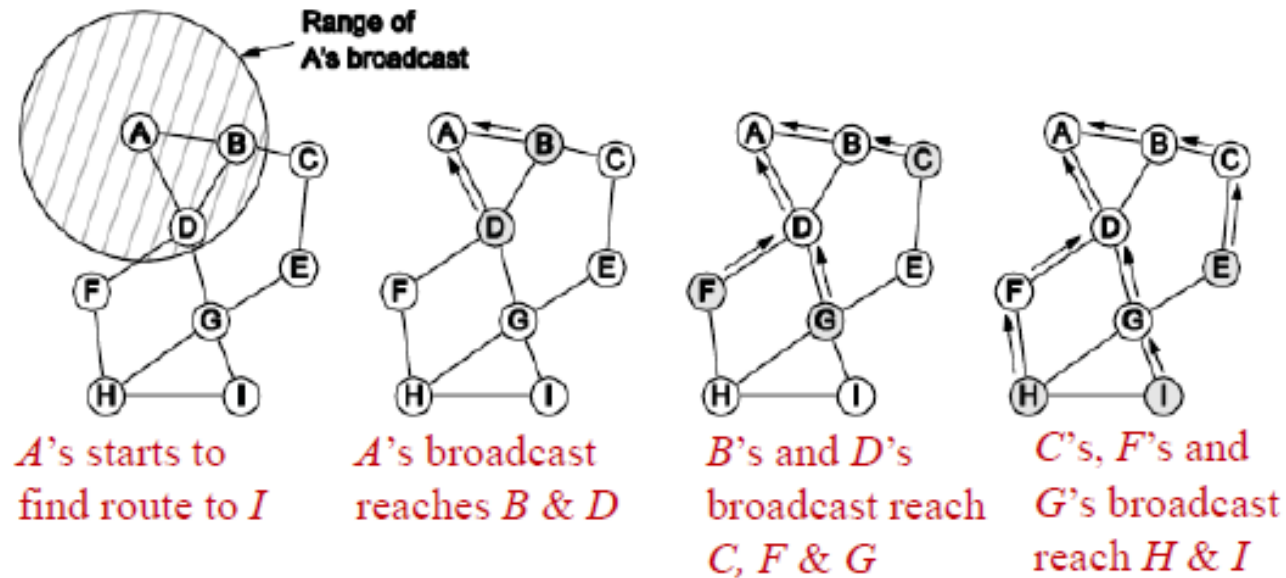
- IPv4: exhaustion of addresses - share a public address
- IPv6: security reasons
- IPv4 - IPv6: NAT-PT

Example: Your friend **A** in Xinjiang wants to reach you **B** (home in Shanghai, now traveling to Yunnan)



Routing in Ad Hoc Networks

- **The network topology changes as wireless nodes move**
 - Routes are computed/constructed on demand.
 - E.g. AODV





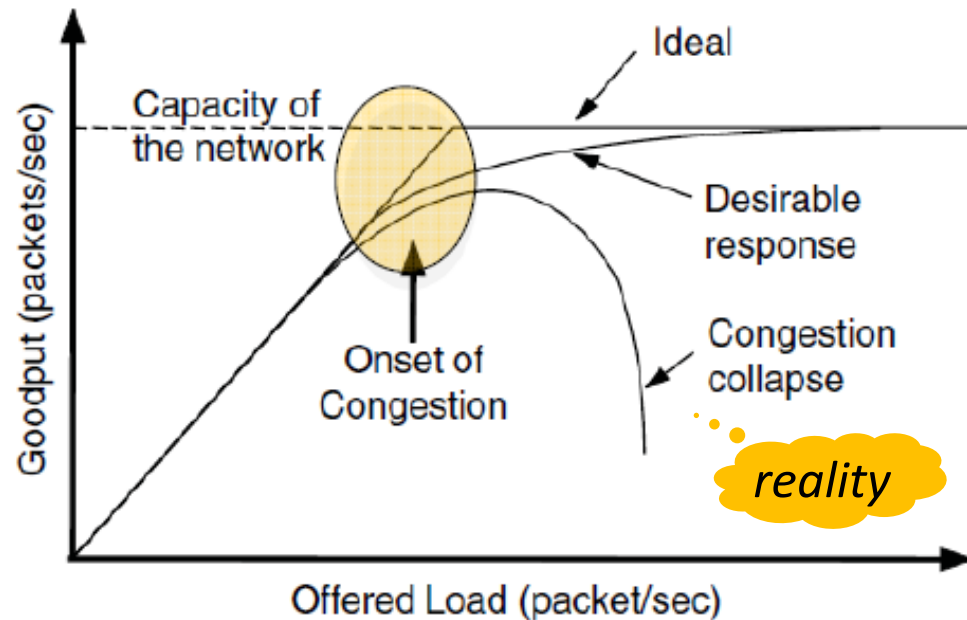
5.3

Congestion Control

Why Congestion Control?

- **Congestion**: When too much traffic is offered, network performance downgrades due to packet loss and/or retransmissions.

Goodput: useful packets (not including retransmission) per unit time.



- **Causes (Why it happens)**: Router receives data faster than it can handle.
 - Insufficient bandwidth
 - Slow hosts
 - Data burst (the same out-going line)
- **Consequences (Not an easy problem)**:
 - Congestion may cause more congestion.
 - Simply moving traffic to another router does not solve the problem.

What is Congestion Control?

- **Different from Flow Control**

- **Flow Control**: handles traffic *over point-to-point links* between a (certain) sender and receiver pair, e.g. a fast host sending to a slow host
- **Congestion Control**: handles traffic *throughout the network*.

- **Two Types of Solutions**

- **Open-loop**: NO runtime feedback, to prevent the problem before it happens.
- **Closed-loop**: Use *feedback* (measurements of the *system performance*) to make corrections *at runtime*.
 - Monitor -> Pass information -> Take actions.
 - Metrics to monitor: % of packets discarded/timeout, avg. queue length, avg. packet delay...

Solution to the Congestion Control Problem

• Congestion Prevention

It's a joint effort – will see more in the next chapter (TCP).

Layer	Policies
Transport	• Retransmission policy • Out-of-order caching policy • Acknowledgement policy • Flow control policy • Timeout determination
Network	• Virtual circuits versus datagram inside the subnet • Packet queueing and service policy • Packet discard policy • Routing algorithm • Packet lifetime management
Data link	• Retransmission policy • Out-of-order caching policy • Acknowledgement policy • Flow control policy

• Congestion Reduction

• Increase Resources (usually bandwidth)

- Use faster lines
- Obtain additional lines
- Utilize alternate pathways
- Utilize “spare” routers

• Decrease Traffic

- Send messages to senders to slow down
- Deny service to some users
- Downgrade service to some/all users
- Schedule usage to achieve better load balance

The Networking Layer Solution

- **Active Queue Management (AQM) at routers**
 - *Queue -> buffers*
 - **Policies:**
 - *Drop-tail: when congestion is detected, drop newly incoming packets.*
 - *Random Early Detection (RED): When congestion is going to happen, randomly select and drop packets in the buffer.*
 - *Early Congestion Notification (ECN)*
 - *Others.*



5.4

Quality of Service (QoS)

Application Requirements Differ

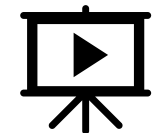
- *Different Applications care about different properties/performances*



Application	Bandwidth	Delay	Jitter	Loss
Email	Low	Low	Low	Medium
File sharing	High	Low	Low	Medium
Web access	Medium	Medium	Low	Medium
Remote login	Low	Medium	Medium	Medium
Audio on demand	Low	Low	High	Low
Video on demand	High	Low	High	Low
Telephony	Low	High	High	Low
Videoconferencing	High	High	High	Low



SSH

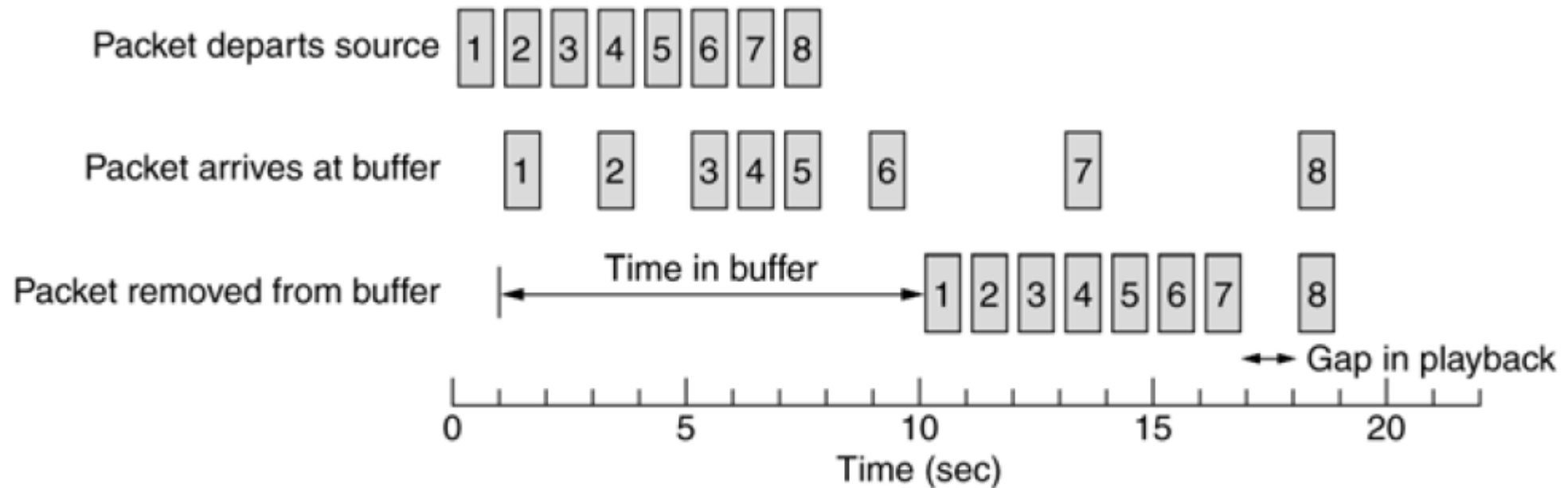


Note: Here “high” means a more demanding requirement, e.g. lower delay

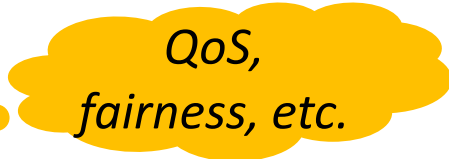
How to satisfy the requirements?

Buffering

- **Jitter**: the variation (standard deviation) in the packets arrival time.
- **Buffers** on the receiving side before deliver: to smooth the jitter.



Scheduling

- *Packets coming from various networks are multiplexed together and queued for transmission at the output buffers associated with a link.*
- **Question**: *Which packet goes out first?* ... 
- **Link-scheduling** discipline: *select the next packet for transmission.*
 - **First-In-First-Out (FIFO)**
 - **Priority Queuing (PQ)**
 - **Round Robin and Weighted Fair Queuing (WFQ)**
 - ...

First-In-First-Out (FIFO)

- **Packets leave the queue in the same order as they arrived.**
- **Queue Dynamics (In Queuing Theory)**

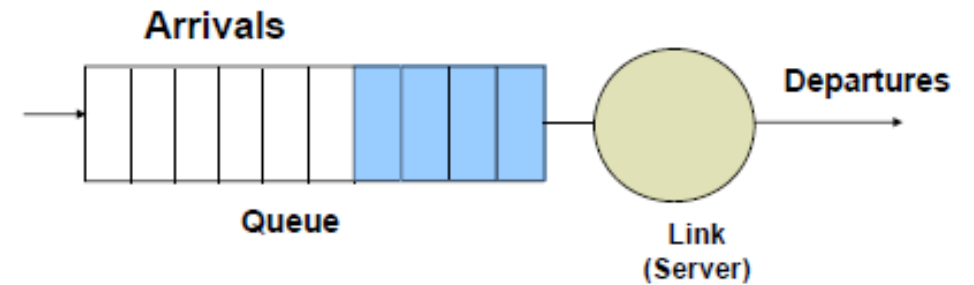
- **Denotations**

- X_n : the amount of traffic arrival in the n^{th} time slot
- Q_n : the queue length at the n^{th} time slot
- C : service rate, max amount of traffic that can be served in one time slot
- B : buffer size

- **Queue Dynamics (Lindley's Recursion)**

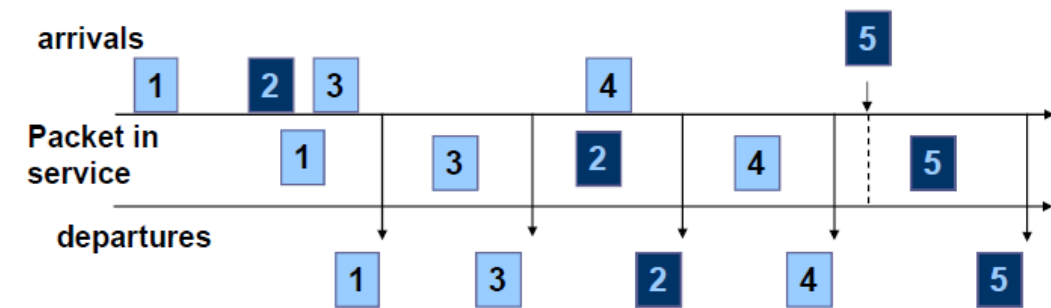
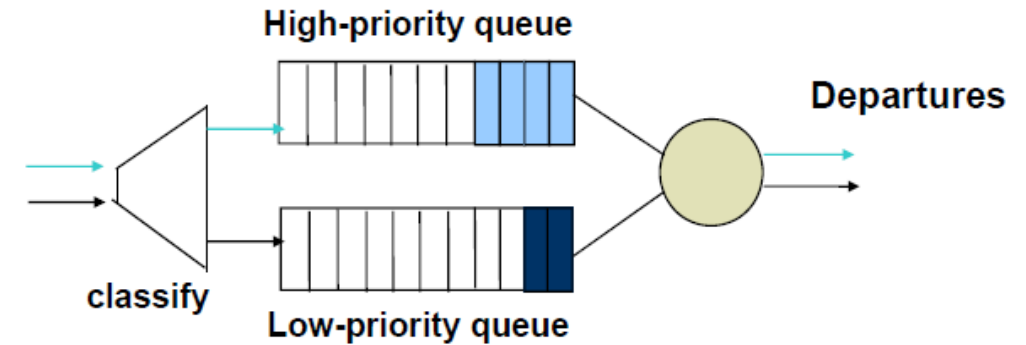
$$Q_{n+1} = \min\{[Q_n + X_n - C]^+, B\} = \min\{\max\{Q_n + X_n - C, 0\}, B\}$$

- **Infinite Buffer ($B = \infty$):** $Q_{n+1} = [Q_n + X_n - C]^+$



Priority Queuing (PQ)

- Packets are classified into two or more **priority classes** at the output queue.
- Each packet has a **priority class** value.
 - Can be explicitly written in the ToS field.
 - Can be determined by values of other fields.
- **Priority Queuing Discipline:**
 - Packets in the same priority class: FIFO
 - Transmit a packet from the highest priority class that has a non-empty queue.

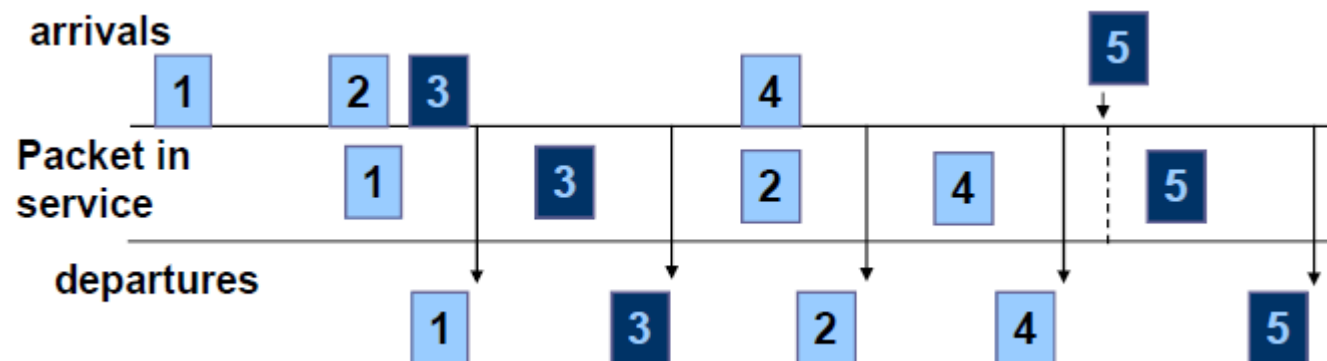


Round Robin Scheduler

- Packets are classified into classes (as in PQ).
- A **Round Robin** Scheduler alternates services among classes. ...
 - A Two-class Example: Transmit a class 1 packet, then a class 2 packet, then another class 1 packet, then another class 2 packet, and so on.
 - **Work-conserving**: The queuing discipline will never allow the link to remain idle whenever there are packets queued for transmission.

Taking turns

Transmit as much as it can.



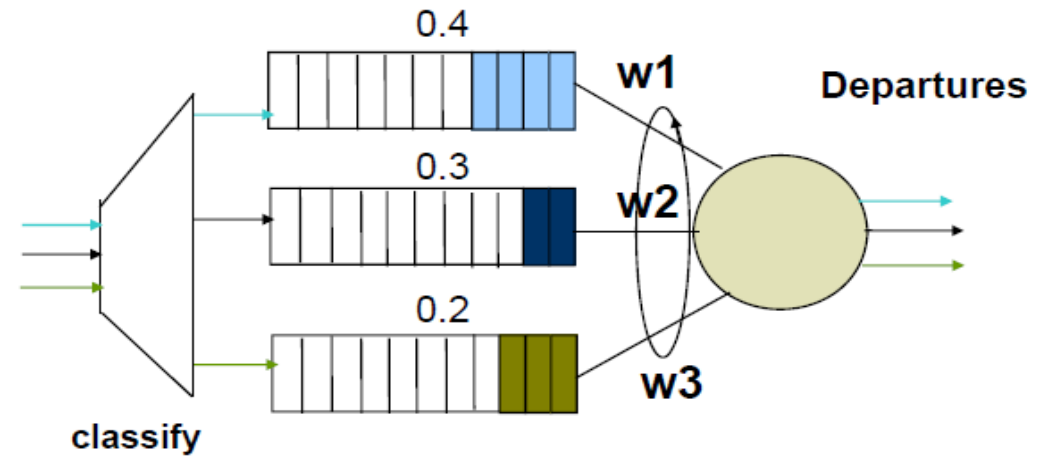
Weighted Fair Queuing (WFQ)

- (basically) A generalized abstraction of Round Robin.

- What is the same: serve in a circular manner

- Differences:

- Each class is assigned a weight w_i .
- During each interval, class i is guaranteed to receive a fraction $(w_i / \sum_j w_j)$ of service.



- Property: For a link of rate R , class i (with packets to send) will receive a throughput of **at least** $R \cdot w_i / \sum_j w_j$.

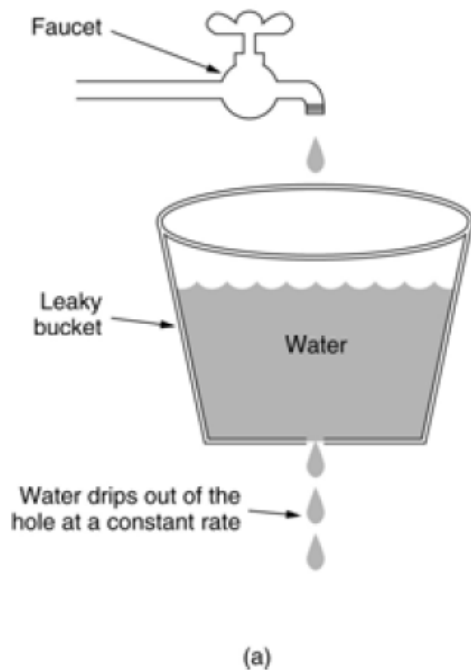
Why?

Traffic Shaping

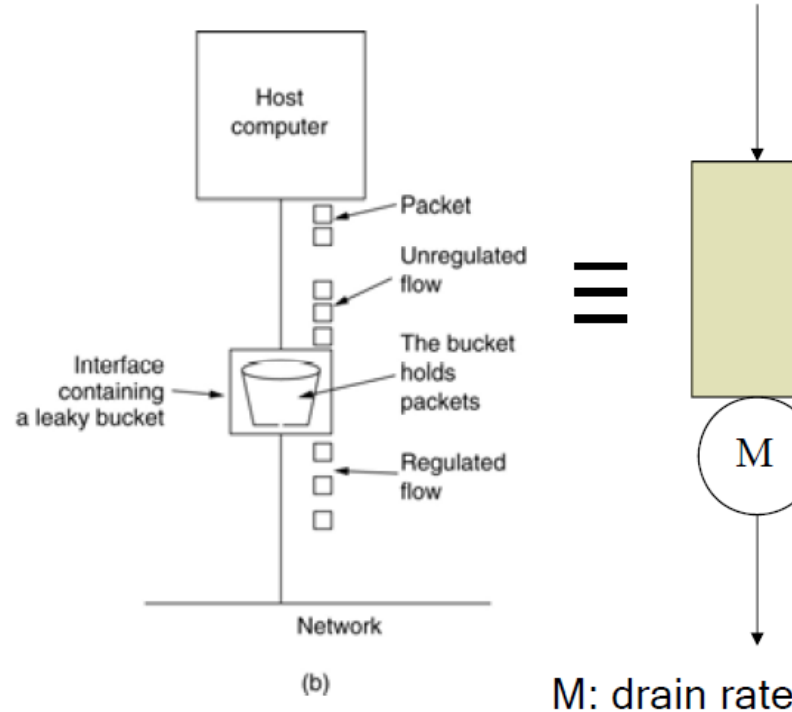
- **Control at the ingress point:** regulate the rate at which a flow is allowed to *inject packets* into the network. – *amount and volume*
- **Traffic Policing:** monitoring a traffic flow.
- **Service Level Agreement (SLA):**  
 - An agreement *between the customers and carriers* on a certain traffic pattern.
 - What's agreed upon: avg. rate, peak rate, burst size, etc.
- **Tools**
 - Leaky bucket
 - Token bucket

The Leaky Bucket Algorithm

- **Leaky Bucket:** there is a max output rate -> to smooth out large bursts



(a) A leaky bucket with water.



(b) A leaky bucket with packets.

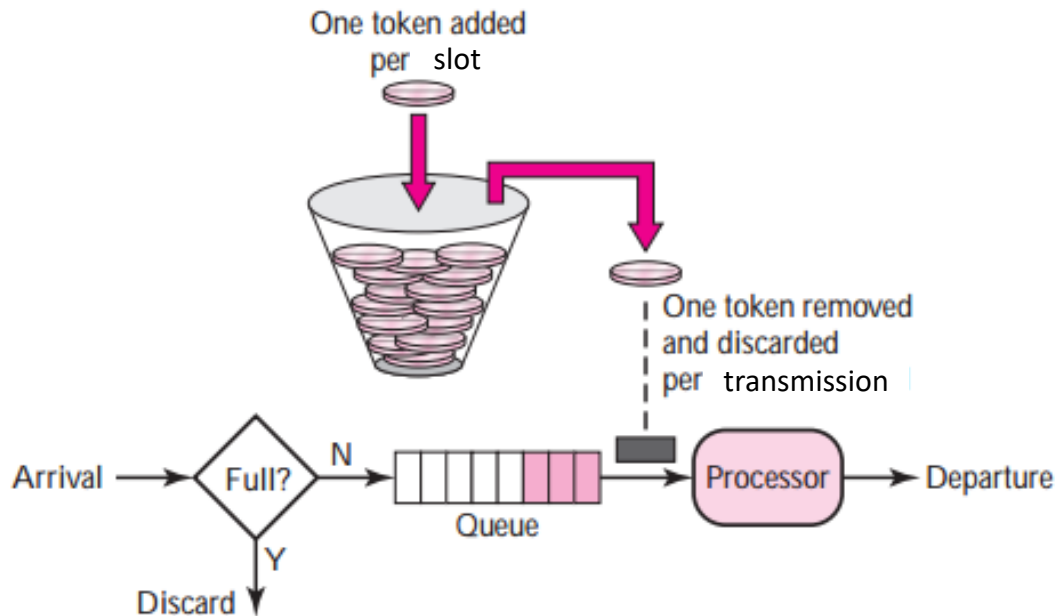
- M : constant service rate
- $X(s, t)$: the amount of traffic generated during time interval $[s, t]$.

- Leaky Bucket: for all s ,
$$X(s, s + t) \leq Mt$$

What if the arrival rate is lower than M ?

The Token Bucket Algorithm

- **Token Bucket:** Each packet has to obtain a token to leave -> to *allow temporary small bursts*.
- **If there is no token in the bucket, the packet must wait for a token.**



- M : (max) output rate (drain rate)
- C : token bucket size
- ρ : token generation rate ($M > \rho$)
- **Token Bucket:** for all s ,

$$X(s, s + t) \leq \min\{Mt, \rho t + C\}$$

Why?

Burst Time/Length in Token Bucket

- **Burst: output at the max rate (M , drain rate)** ...

Recall $M > \rho$

- **Burst time/length S : the lasting time of a burst.**

- M : drain rate

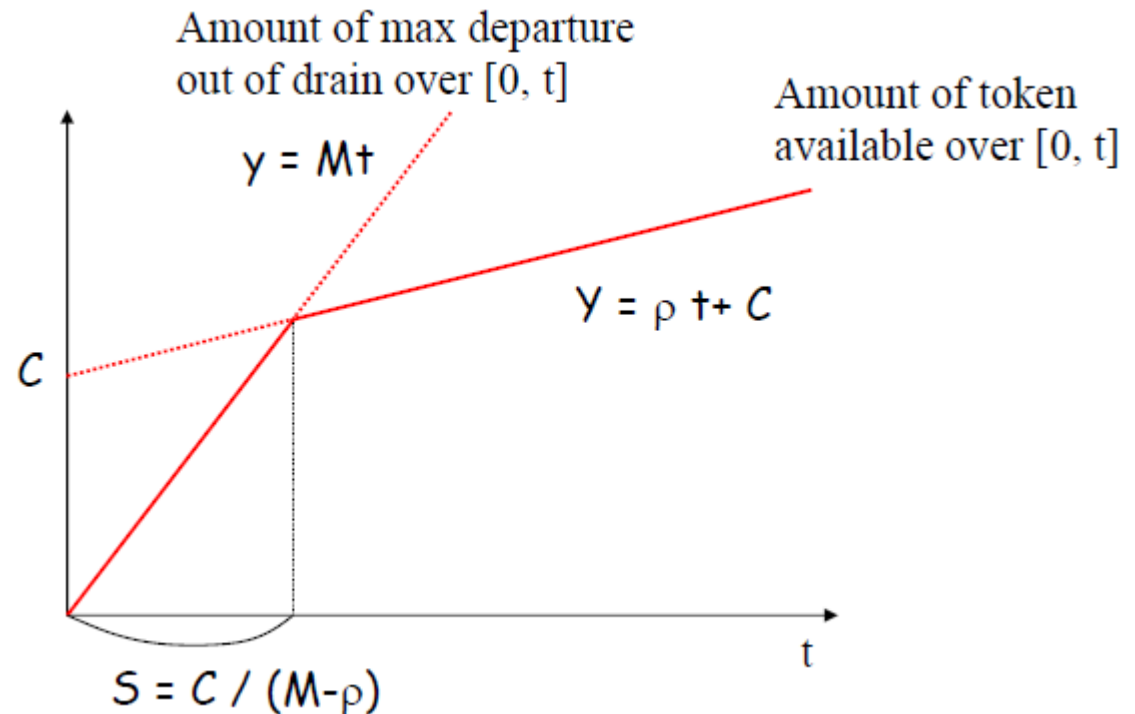
- C : token bucket size

- ρ : token rate

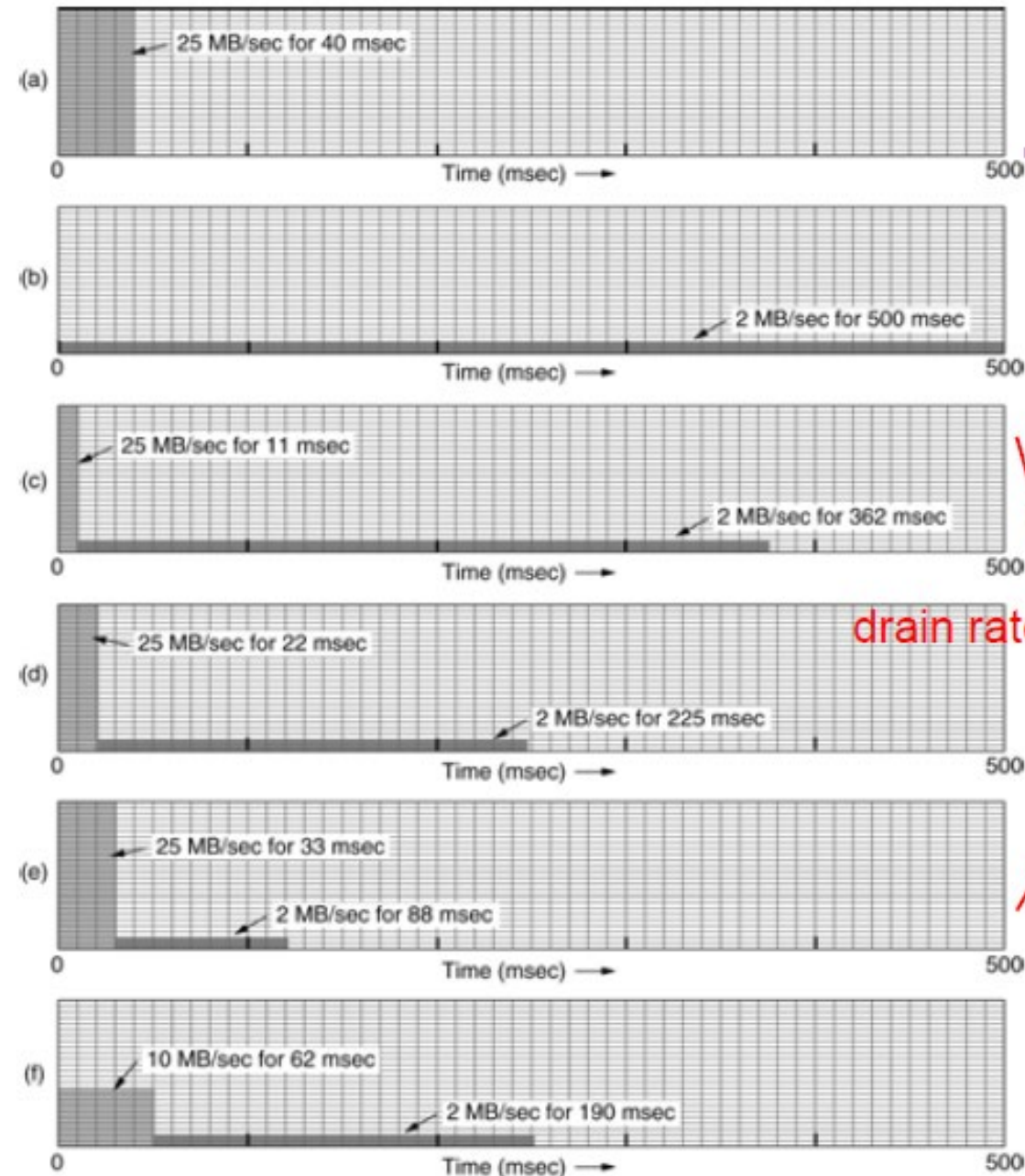
- $\rho S + C = MS \Rightarrow S = \frac{C}{M - \rho}$

- E.g. $C = 250 \text{ KB}$, $M = 25 \text{ MB/s}$,

$$\rho = 2 \text{ MB/s}, S = \frac{C}{M - \rho} = 11 \text{ ms}$$



An Example: Leaky Bucket and Token Bucket



- (a) Input to a leaky bucket.
- (b) Output from a leaky bucket. $M = 2\text{MB/sec}$

Output from a token bucket with sizes of

(c) 250 KB, (d) 500 KB, (e) 750 KB, . . .

$$\rho = 2\text{MB/sec}$$

- (f) Output from a 500KB token bucket feeding a $M = 10\text{MB/sec}$ leaky bucket.

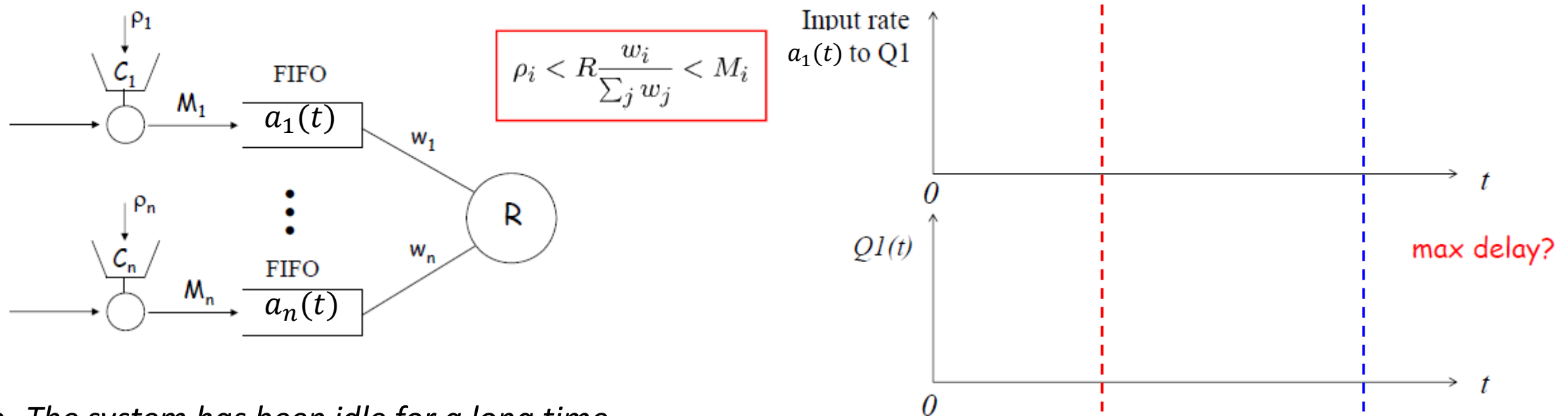
*Smooth out,
no burst.*

*Burst of different
lengths*

*Combine the
two, and
possibly more*

Ex.5.4 Traffic Shaping and Scheduling

Consider a traffic shaping system depicted as the following.



- The system has been idle for a long time.
- At time $t=0$, a large amount of packets arrive in all classes.

Find the maximum delay of a packet in the class-1 queue.

Admission Control

- **QoS Routing:** Reservations made along the route.
- **Flow Negotiating:** Application and routers agree on a set of flow parameters, which is called a **flow specification**.

Parameter	Unit
Token bucket rate ρ	Bytes/sec
Token bucket size C	Bytes
Peak data rate M	Bytes/sec
Minimum packet size	Bytes
Maximum packet size	Bytes

Service Models in the Internet

- **Best effort model**

- Every user receives the same level of service.
- Congestion results in severe degradation to applications requiring some min amount of bandwidth/delay.

- **Integrated services model**

- Requires resources (bandwidth/buffers) to be explicitly reserved for each traffic flow
-> RSVP signaling protocol
- Requires packet classification, packet scheduling and CAC (call admission control)

- **Differentiated services (DiffServ) model**

- Scalable and flexible services based on “class” of traffic

Summary

- **Functions of the Network Layer: routing from the source to destination**
 - **Addressing:** IP addresses, subnetting, mask, routing table configuration
 - **Routing:** protocols, algorithms
 - Link State Routing, e.g., shortest path routing Dijkstra algorithm
 - Distance Vector Routing, e.g., Bellman-ford
 - Hierarchical Routing, intra-AS, inter-AS
 - **Congestion Control:** a joint effort – mostly in TCP (transport)
 - **QoS:** requirements from the applications
 - Queue: FIFO, PQ
 - Scheduling: Round Robin, WFQ